

Sugestões de resolução de incidentes através do uso de LLM's

João Sousa

**Dissertação para obtenção do Grau de Mestre em
Engenharia Informática, Área de Especialização em
Engenharia de Software**

**Orientador: Paulo Gandra Sousa
Supervisor: João Peixoto**

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, Porto, 20 de setembro de 2026

Resumo

O presente documento foi elaborado com o intuito de descrever o trabalho de investigação realizado com base numa proposta de solução apresentada pela empresa *Critical Techworks* (CTW).

A proposta consiste em desenvolver uma ferramenta capaz de ajudar os membros de equipas de suporte, no decorrer do seu dia a dia. Através dessa ferramenta, pretende-se fornecer funcionalidades como: correlação de incidentes, sumarização de descrições em linguagem natural, sugestões de mitigação e alertas relacionados como informação adicional aos incidentes e o objetivo de diminuir os tempos médios de resposta.

Este trabalho de investigação, é apenas uma primeira parte onde é definido o problema, os objetivos e é delineado o planeamento para o desenvolvimento da solução. Para além disso, é realizada uma análise científica e das soluções que já existem no mercado de forma a contextualizar melhor o problema em questão.

Para concluir, a investigação foi bem sucedida, uma vez que, a questão de investigação foi respondida, concluindo que o desenvolvimento da solução é viável e que as ferramentas de *LLM* podem ajudar a alcançar os objetivos pretendidos.

Palavras-chave: Modelo de Linguagem de Grande Escala, Inteligência Artificial, Processamento da Linguagem Natural, Incidente, Mitigação, Sugestão

Abstract

This document was prepared to describe the research work carried out based on a solution proposal presented by the company Critical Techworks (CTW).

The proposal was based on the development of a tool capable of helping support team members in their day-to-day work. It provides features such as incident correlation, summarization of descriptions in natural language, mitigation suggestions, and related alerts as additional information on incidents, with the aim of reducing average response times.

This research is only the first part, in which the problem and objectives are defined and the plan for developing the solution is outlined. In addition, a scientific analysis of existing solutions on the market is carried out in order to better contextualize the problem at hand.

In conclusion, the research was successful, with the work answering the research questions and concluding that the development of the solution is feasible and that LLM tools can help achieve the desired objectives.

Keywords: Large Language Model, Artificial Intelligence, Natural Language Processing, Incident, Mitigation, Suggestion

Agradecimentos

Gostaria primeiramente de agradecer ao meu orientador de estágio, o Doutor Paulo Gandra Sousa, professor coordenador do Instituto Superior de Engenharia do Porto, pela sua constante disponibilidade, apoio e conselhos que tornaram o projeto muito mais enriquecedor.

Um agradecimento também ao supervisor da empresa João Peixoto, o melhor (e único) *Chief Technical Titan* que tive, não só pela confiança depositada em mim, mas também pelo acompanhamento e suporte prestado ao longo do meu percurso profissional na empresa.

Um obrigado em especial a todos os professores que me acompanharam, por todo o esforço que tiveram em transmitir os seus conhecimentos, tornando possível a realização dos meus objetivos profissionais e académicos, e ao ISEP por disponibilizar a estrutura para tudo isso poder acontecer.

Fica também um forte agradecimento ao meu colega universitário, João Brito, pelo companheirismo e disponibilidade para que, juntos, superássemos os mais variados obstáculos encontrados pelo caminho até este momento.

Agradecer especialmente à minha namorada, que me apoiou desde a minha entrada na vida universitária, pelos conselhos dados e pelos domingos bem passados a estudar, fazer projetos e a redigir este documento.

Por fim, a todos os meus colegas, amigos e família que sempre prestaram um enorme apoio, o meu imenso obrigado.

Conteúdo

Lista de Figuras	xv
Lista de Tabelas	xvii
Lista de Código	xix
Lista de Acrónimos	xxi
1 Introdução	1
1.1 Organização e Contexto	1
1.2 Problema	1
1.3 Objetivos	2
1.4 Planeamento	2
1.5 Considerações Éticas	3
1.6 Estrutura do Documento	4
2 Fundamentação Teórica	5
2.1 <i>Large Language Model (LLM)</i>	5
2.2 Prompt Engineering	5
2.3 <i>Retrieval-Augmented Generation (RAG)</i>	6
2.3.1 Estratégias de Recuperação em Sistemas <i>RAG</i>	6
2.4 Fine-tuning	7
2.5 Métricas de avaliação	7
3 Estado da Arte	9
3.1 Metodologia de Investigação	9
3.1.1 Pergunta(s) de Investigação	9
3.1.2 Metodologia PICOC(T)	9
3.1.3 Palavras-Chave	10
3.1.4 Critérios de Inclusão e Exclusão	11
3.1.5 PRISMA	12
3.2 Revisão de Literatura	13
3.2.1 Análise de Causa Raiz e Correlação de Incidentes	13
3.2.2 Sugestões de Mitigação e Uso de Incidentes Anteriores	14
3.2.3 Apoio à Contextualização de Incidentes com <i>LLMs</i>	15
3.2.4 Técnicas e Modelos para Análise de Incidentes	15
3.2.5 Conclusão	16
3.3 Soluções no Mercado	17
3.3.1 <i>AWS AIOps</i>	17
3.3.2 <i>New Relic AIOps</i>	17
3.3.3 <i>Atlassian Ops Guide Agent</i>	18

3.3.4	<i>Elastic AI Assistant</i>	18
3.3.5	Conclusão	18
4	Análise	21
4.1	Desenho da Solução	21
4.2	Dataset	22
4.2.1	Estrutura	22
4.2.2	Origem dos Incidentes	23
4.2.3	Destino dos Incidentes	25
4.2.4	Conclusão	28
4.3	Engenharia de Requisitos	28
4.3.1	Requisitos Funcionais	28
4.3.2	Requisitos Não Funcionais	29
4.4	Conclusão	30
5	Desenho	31
5.1	Diagrama de Implantação	31
5.2	Diagrama de Sequência	31
5.3	Conclusão	32
6	Desenvolvimento	33
6.1	Stack Tecnológica	33
6.1.1	Python	33
6.1.2	RAG	34
6.1.3	Langfuse	34
6.2	Metodologia de Implementação	35
6.2.1	Spec Driven Development	35
6.2.2	Limitações	35
6.2.3	Agentic <i>Spec Driven Development</i> (<i>SDD</i>)	36
6.2.4	Specs	37
6.3	Estrutura	38
6.4	Implementação	40
6.4.1	Integração com ITSM	40
6.4.2	Integração com Confluence	41
6.4.3	Integração com LLM	42
6.4.4	Considerações de Implementação	43
	RAG vs Fine Tuning	43
	Dados Sensíveis	44
	Custos e Tempo de Execução	44
6.5	Conclusão	45
7	Validação de Resultados	47
7.1	Metodologia	47
7.2	Análise dos resultados	48
7.2.1	Avaliação dos Resumos	49
7.2.2	Avaliação das Sugestões de Mitigação	50
7.2.3	Avaliação dos Incidentes e Páginas Relacionadas	51
7.2.4	Avaliação da Qualidade do serviço	52
7.2.5	Avaliação dos Modelos LLM	54
7.2.6	Avaliação do Feedback da Equipe	54

7.3	Conclusão	54
8	Conclusão	55
8.1	Trabalho Realizado	55
8.2	Limitações	55
8.3	Trabalho Futuro	55
8.4	Apreciação Final	55
	Bibliografia	57
	Apêndice A Registos de incidentes	59

Lista de Figuras

1.1	WBS Detalhado	3
1.2	Diagrama de <i>Gantt</i>	3
2.1	Arquitetura <i>RAG</i>	6
3.1	Diagrama <i>PRISMA</i>	13
4.1	Incidentes Manuais vs Automáticos	24
4.2	Percentagem de Incidentes Manuais vs Automáticos	24
4.3	Incidentes Redirecionados	25
4.4	Percentagem de Incidentes Redirecionados	26
4.5	Incidentes Manuais Redirecionados	27
4.6	Percentagem de Incidentes Manuais Redirecionados	27
5.1	Diagrama de Implantação	31
5.2	Diagrama de Sequência	32
6.1	Modelo Base de uma Especificação	37
6.2	Modelo Base de uma Especificação	39
7.1	Exemplo de uma <i>Golden Reference</i>	48
7.2	Duração para Selecionar Documentação Relevante	53
A.1	Detalhes do incidente	59
A.2	Detalhes do incidente	60
A.3	Detalhes do incidente	60
A.4	Detalhes do incidente	61

Lista de Tabelas

3.1	Metodologia PICOCT	10
3.2	Repositórios de Pesquisa	11
3.3	Critérios de Inclusão	12
3.4	Critérios de Exclusão	12
3.5	Funcionalidades de Soluções Existentes	19
3.6	Técnicas usadas por Soluções Existentes	19
4.1	Campos Utilizados do Dataset	22
4.2	Campos Descartados do Dataset	23
4.3	Requisitos Funcionais	29
4.4	Requisitos não Funcionais	30
7.1	Campos Utilizados do Dataset	47
7.2	Incidentes na <i>Golden Reference</i>	49
7.3	Critérios de Avaliação dos Resumos	49
7.4	Avaliação dos Resumos Gerados	49
7.5	Critérios de Avaliação das Sugestões	50
7.6	Avaliação das Sugestões Geradas	50
7.7	Avaliação das Sugestões Geradas (Exclui não Correspondências)	51
7.8	Critérios de Avaliação dos Relacionados	51
7.9	Avaliação dos Incidentes Relacionados	51
7.10	Avaliação das Páginas Relacionadas	51
7.11	Critérios de Avaliação de Latência e Custos	52
7.12	Avaliação de Latência	52
7.13	Avaliação de Custos	52
7.14	Comparação de Latência e Custo sem <i>Confluence</i>	54

Lista de Código

3.1	Termos de Pesquisa	11
-----	------------------------------	----

Lista de Acrónimos

ACM	<i>Association for Computing Machinery.</i>
API	<i>Interface de Programação de Aplicações.</i>
ASE	<i>Automated Software Engineering.</i>
AUC	<i>Area Under the Curve.</i>
BMW	<i>Bayerische Motoren Werke.</i>
CTW	<i>Critical Techworks.</i>
CoT	<i>Chain-of-Thought.</i>
DBSCAN	<i>Density-Based Spatial Clustering of Applications with Noise.</i>
DDD	<i>Domain-Driven Design.</i>
DSL	<i>Domain-Specific Language.</i>
FSE	<i>ACM International Conference on the Foundations of Software Engineering.</i>
GNNs	<i>Graph Neural Networks.</i>
ICSE	<i>International Conference on Software Engineering.</i>
ITSM	<i>IT Service Management.</i>
LLM	<i>Large Language Model.</i>
ML	<i>Machine Learning.</i>
NSPE	<i>National Society of Professional Engineers.</i>
OCEs	<i>On-Call Engineers.</i>
PRISMA	<i>Preferred Reporting Items for Systematic reviews and Meta-Analyses.</i>
RAG	<i>Retrieval-Augmented Generation.</i>
ROUGE	<i>Recall-Oriented Understudy for Gisting Evaluation.</i>
RQ	<i>Research Question.</i>
SDD	<i>Spec Driven Development.</i>
SMS	<i>Systematic Mapping Study.</i>
TSG	<i>Troubleshooting Guide.</i>
TTM	<i>Time to mitigate.</i>
WBS	<i>Work Breakdown Structure.</i>
IA	Inteligência Artificial.
IPP	Instituto Politécnico do Porto.
ISEP	Instituto Superior de Engenharia do Porto.
PICOCT	População, Intervenção, Comparação, Outcome, Contexto e Tipo de estudo.
PLN	Processamento de Linguagem Natural.
PREPD	Preparação para Dissertação.

TI	Tecnologias de Informação.
TMR	Tempo Médio de Resposta.

Capítulo 1

Introdução

Este capítulo apresenta a organização responsável pela ideia do tema a ser desenvolvido, o contexto do seu surgimento e os problemas que a originaram. São abordados os objetivos, o planeamento e as considerações éticas tidas em conta para executar a proposta. Por último, é descrita a estrutura deste documento de forma a facilitar a leitura do mesmo.

1.1 Organização e Contexto

A *Critical Techworks (CTW)* nasceu de um empreendimento conjunto entre a *Critical Software* e o grupo *Bayerische Motoren Werke (BMW)*. Estabelecida em 2018, esta empresa desenvolve *software* exclusivamente para os produtos da *BMW*. A sua missão é "transformar a forma como o mundo se move" focando-se para isso na transformação digital com as melhores práticas do mercado.

A organização estrutura-se em equipas ágeis multidisciplinares, agrupadas em unidades e domínios tecnológicos, o que promove autonomia, ciclos de desenvolvimento rápidos e colaboração contínua. Esta abordagem permite alinhar o desenvolvimento de *software* com necessidades reais dos veículos e serviços digitais da *BMW*.

O presente relatório foi desenvolvido no âmbito da unidade curricular de Preparação para Dissertação (PREPD) do Instituto Superior de Engenharia do Porto (ISEP), com o intuito de demonstrar a viabilidade de uma solução para o problema em causa.

1.2 Problema

A análise e resolução de incidentes está no dia a dia das equipas que lidam com projetos potencialmente críticos. Um incidente é um evento que provoca interrupção ou degradação num qualquer serviço, e podem comprometer significativamente a disponibilidade dos serviços e resultar em perdas financeiras substanciais (J. Jiang et al. 2020). Estes incidentes podem ter por origem duas principais fontes: automático, através de métricas definidas pelas equipas; e manuais, criados por clientes das aplicações desenvolvidas. Um registo de incidente (*ticket*) é um documento, normalmente gerido por um sistema externo (como *ServiceNow* ou *Jira Service Management*), onde é possível visualizar detalhes do incidente, tais como a sua descrição e prioridade, e onde um elemento da equipa pode descrever os passos de resolução efetuados.

Na área de Tecnologias de Informação (TI), as equipas podem manter vários serviços em simultâneo e por essa razão os membros da equipa de suporte (responsáveis por manter os serviços estáveis 24x7) podem ficar sobrecarregados de incidentes, fazendo com que tenham

de lidar com eventos provenientes de vários serviços paralelamente. Numa situação como esta, os membros da equipa de suporte podem ter um sentimento de confusão, misturando temas e errando nas suas análises, o que pode causar piores resoluções de incidentes e o aumento do tempo para mitigação (*Time to mitigate (TTM)*). Outro cenário frequente para os *On-Call Engineers (OCEs)* envolve incidentes que ocorrem fora do horário de expediente, especialmente durante a madrugada, quando o membro da equipa de suporte pode não estar na sua plena capacidade de raciocínio e operacional.

Foi também referido anteriormente os incidente com origem manual. Estes, são criados por clientes finais e/ou equipas de testes e podem conter mais ou menos informação. Muitas vezes, os *OCEs* necessitam de realizar uma análise mais extensa para compreender o conteúdo do incidente e a resolução mais eficaz a aplicar.

Uma interpretação ou resolução inadequada das ocorrências pode reduzir a disponibilidade dos serviços, o que, por sua vez, pode gerar prejuízos significativos para a organização, seja pela perda de receitas, seja pelo impacto negativo na experiência do utilizador final.

1.3 Objetivos

O objetivo deste projeto consiste em solucionar os problemas demonstrados na ponto anterior. Assim, a solução deve ser capaz de:

- Utilizar contexto histórico, como incidentes anteriores e documentação dos projetos;
- Gerar resumos de um incidente específico;
- Gerar sugestões de investigação e mitigação;
- Informar incidentes relacionados com o incidente em questão.

A fim de alcançar estes objetivos, foi definido um conjunto de passos necessários para organizar o trabalho desenvolvido. Uma metodologia de investigação foi selecionada para ajudar no processo de categorização e análise de literatura relevante ao tema em estudo, que serviu de base para o desenvolvimento do projeto proposto.

Espera-se que, com a geração de textos em linguagem natural, se consiga uma resposta mais rápida e consistente, o que se reflete numa redução do Tempo Médio de Resposta (TMR) aos incidentes. Assim, a carga de trabalho associada a operações por parte dos membros da equipa de suporte envolvidos diminui, libertando-os para o desenvolvimento de novas funcionalidades. De igual forma, é esperado que a diminuição do TMR possa aumentar a disponibilidade das aplicações, o que traz um impacto direto ao cliente final. Adicionalmente, a utilização de informação histórica e documentação procura ir ao encontro dos requisitos das diferentes partes interessadas, apoiando a tomada de decisão e permitindo que as equipas dediquem mais tempo a outras atividades de valor.

1.4 Planeamento

O planeamento do trabalho foi definido com base numa *Work Breakdown Structure (WBS)*, que serviu de base à construção do diagrama de *Gantt* (Figura 1.1 e 1.2). A *WBS* permite estruturar o projeto em três fases distintas, cada uma com as respetivas tarefas e entregáveis. Por sua vez, o diagrama de *Gantt* é utilizado para calendarizar as atividades, identificar dependências entre tarefas e definir marcos de controlo.

	WBS	Nome	Nível WBS	Duração	Início	Fim
1	1	Projeto Dissertação	Projeto	184 dias	01/10/25...	15/06/26...
2	1.1	Planeamento	Fase	11 dias	01/10/25...	15/10/25...
3	1.1.1	Formalização do Planeamento	Entregável	11 dias	01/10/25, ...	15/10/25, ...
4	1.2	Preparação	Fase	60 dias	16/10/25...	07/01/26...
5	1.2.1	Project Charter	Entregável	11 dias	16/10/25, ...	30/10/25, ...
6	1.2.2	Extended Abstract	Entregável	32 dias	31/10/25, ...	15/12/25, ...
7	-----	Primeira Entrega	Milestone	0 dias	15/12/25, ...	15/12/25, ...
8	1.2.3	Documento Final da Prepação	Entregável	17 dias	16/12/25, ...	07/01/26, ...
9	1.2.4	Apresentação	Entregável	17 dias	16/12/25, ...	07/01/26, ...
10	-----	Segunda Entrega	Milestone	0 dias	07/01/26, ...	07/01/26, ...
11	1.3	Desenvolvimento	Fase	113 dias	08/01/26...	15/06/26...
12	1.3.1	Protótipo da Solução	Entregável	92 dias	08/01/26, ...	15/05/26, ...
13	1.3.2	Documento Final	Entregável	113 dias	08/01/26, ...	15/06/26, ...
14	1.3.3	Apresentação Final	Entregável	21 dias	18/05/26, ...	15/06/26, ...
15	-----	Última Entrega	Milestone	0 dias	15/06/26, ...	15/06/26, ...

Figura 1.1: WBS Detalhado

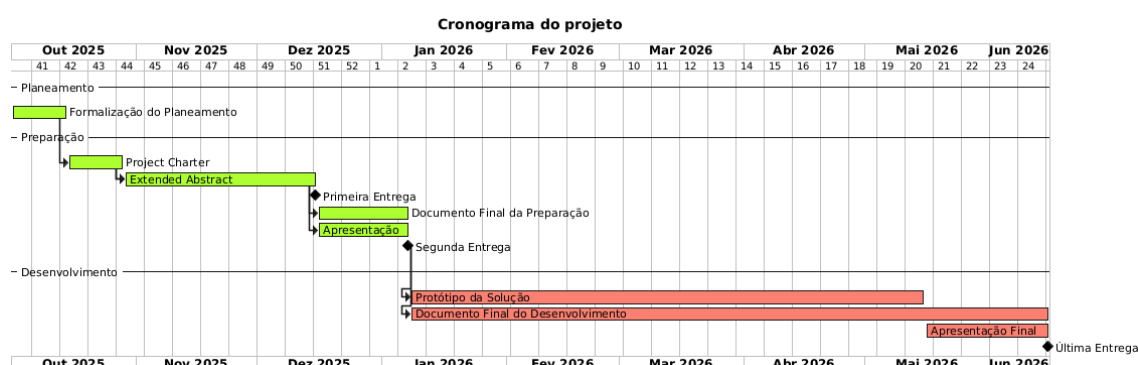


Figura 1.2: Diagrama de Gantt

Este planeamento assegura a ligação entre os objetivos do projeto, os entregáveis previstos e o respetivo calendário, facilitando o acompanhamento do progresso ao longo de todo o seu desenvolvimento.

1.5 Considerações Éticas

Neste projeto, foram seguidos os princípios do Código de Ética e Conduta Profissional da *Association for Computing Machinery (ACM)*, do Código de Ética para Engenheiros da *National Society of Professional Engineers (NSPE)* e do Código de Engenharia de Software da *ACM/IEEE-CS*, de forma a orientar o estudo e a atuar com responsabilidade, tanto em relação aos dados como às pessoas afetadas pela solução.

Tendo em conta que os dados a serem utilizados podem conter informações confidenciais, foi avaliada a possibilidade de os anonimizar e, caso não seja satisfatório para as partes interessadas, será garantido que apenas os resultados agregados sejam apresentados, evitando quaisquer riscos de exposição de dados confidenciais. Para além disso, não foram identificados outros riscos éticos relevantes no projeto.

Para além dos códigos acima mencionados, foi também seguido o Código de Boas Práticas e de Conduta do Instituto Politécnico do Porto (IPP).

Durante o desenvolvimento da solução foi utilizada Inteligência Artificial como apoio à implementação do código. Esta utilização, em conjunto com a abordagem *Spec Driven Development* (SDD), permitiu definir previamente as especificações e requisitos de cada componente antes da sua implementação. O código gerado pela IA foi analisado e validado pelo autor, não sendo utilizado de forma automática sem revisão. Desta forma, a responsabilidade pelas decisões tomadas e pelo código integrado na solução manteve-se sempre do lado do autor.

Quanto às competências necessárias, o autor é também um dos muitos membros de equipa de suporte que lidam com o tipo de problema que este projeto aborda, estando também a aprofundar os conceitos no mestrado que originou esta investigação. Essa combinação de experiência na área e aprendizagem académica ajuda a assegurar que o trabalho é feito com rigor, ética e atenção às melhores práticas profissionais.

1.6 Estrutura do Documento

O relatório encontra-se dividido em sete capítulos que evidenciam os passos progressivos no desenvolvimento da solução:

- **Capítulo 1 - Introdução:** Explora os problemas e objetivos da projeto;
- **Capítulo 2 - Estado da Arte:** Apresenta a metodologia de investigação utilizada, a revisão de literatura e as soluções semelhantes à que se pretende desenvolver;
- **Capítulo 3 - Analise:** Aborda a solução proposta, o conjunto de dados utilizados e a engenharia de requisitos do problema;
- **Capítulo 4 - Desenho:** Adiciona algum contexto ao funcionamento da solução através de diagramas de implementação e sequência;
- **Capítulo 5 - Implementação:** Apresenta detalhes do desenvolvimento da solução, como a identificação das ferramentas utilizadas, metodologia aplicada, a estrutura e decisões de negócio;
- **Capítulo 6 - Validação de Resultados:** Descreve os resultados obtidos assim com a sua avaliação nas diferentes métricas obtidas;
- **Capítulo 7 - Conclusão:** Conclui o desenvolvimento do projeto evidenciando os resultados obtidos e os objetivos inicialmente propostos de forma a concluir o sucesso do projeto;
- **Anexo A - Incidente:** Apresenta o exemplo de um incidente real;
- **Anexo B - Prompts:** ;
- **Anexo C - Especificações:** ;
- **Anexo D - Agentes:** ;

Capítulo 2

Fundamentação Teórica

Remover este capítulo e passar info para estado da arte..

Este capítulo apresenta os principais conceitos teóricos necessários para compreender o trabalho desenvolvido. Começa-se por uma breve introdução aos *Large Language Models* (LLMs), incluindo as suas principais limitações e as arquiteturas disponíveis para os adaptar, como o Prompt Engineering, o *Retrieval-Augmented Generation* (RAG) e o *Fine-tuning*. Por fim, são apresentadas as métricas utilizadas para avaliar a qualidade, relevância, custo e tempo de resposta da solução.

2.1 Large Language Model (LLM)

Os LLMs são modelos de linguagem que são treinados com grandes quantidades de dados, baseados em arquiteturas de transformadores, com capacidade de entender e gerar linguagem de fácil compreensão. Estes modelos conseguem desempenhar tarefas como geração de resumos, extração de informações e resposta a perguntas, tornando-os particularmente relevantes para cenários em que a informação é maioritariamente textual e heterogênea, como é o caso da gestão de incidentes em TI (Goel et al. 2024).

No entanto, a utilização de LLMs apresenta limitações conhecidas, incluindo o fenómeno de *hallucination* (geração de informação plausível mas incorreta) e forte dependência da qualidade do contexto fornecido. Embora estes modelos demonstrem elevada capacidade de generalização, o seu desempenho piora significativamente quando o contexto é insuficiente ou o problema não está claramente estruturado, tornando fundamental a definição cuidada do contexto e das instruções nos cenários operacionais (Roy et al. 2024).

2.2 Prompt Engineering

Um prompt é uma instrução, comando ou pergunta que se dá a um sistema tecnológico para que este execute uma tarefa específica.

Já o *prompt engineering* consiste na criação de instruções estruturadas fornecidas aos modelo para orientar as suas respostas, permitindo melhorar significativamente a qualidade da resposta. Com um *prompt* bem estruturado é possível aumentar a precisão e a consistência das respostas em tarefas de análise textual e síntese de informação (Zhang et al. 2024).

Esta abordagem é relevante em ambientes de suporte operacional, onde a definição cuidadosa do formato da resposta reduz a ambiguidade e facilita a integração automática com sistemas externos. Ainda assim, o *prompt engineering* não elimina completamente as limitações dos

LLMs, sendo frequentemente combinado com outras técnicas como *RAG* ou *Fine-Tuning* para alcançar resultados mais confiáveis.

2.3 RAG

Em vez de depender apenas do conhecimento que já está presente no modelo, o sistema pode procurar primeiro informação relevante numa fonte de dados externa e usar essa informação como contexto para gerar a resposta. Esta abordagem é conhecida como *RAG* e é especialmente útil em situações em que a informação muda frequentemente e é importante garantir respostas corretas (Goel et al. 2024).

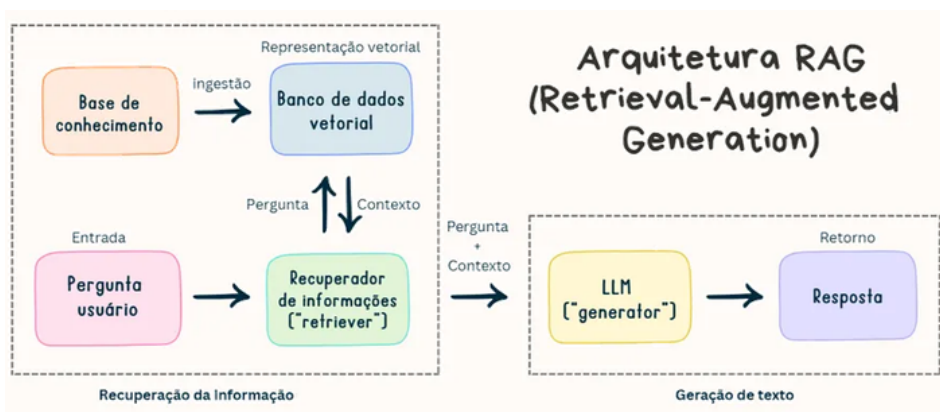


Figura 2.1: Arquitetura *RAG*

Entre as principais vantagens desta arquitetura está a possibilidade de obter respostas mais relevantes e corretas, uma vez que o modelo passa a ter acesso a informação específica em vez de depender apenas do conhecimento que já possui. Outra vantagem é que a informação pode ser atualizada sem ser necessário voltar a treinar o modelo. No entanto, esta abordagem também tem algumas limitações. A qualidade da resposta depende da qualidade e da relevância dos documentos encontrados. Se forem recuperados documentos pouco relevantes ou se faltar informação importante, a resposta do modelo pode não ser útil. Além disso, um sistema de *RAG* requer uma base de documentos bem organizada e implica ter em conta o tempo necessário para pesquisar a informação e os recursos utilizados (Goel et al. 2024; Roy et al. 2024).

2.3.1 Estratégias de Recuperação em Sistemas RAG

Tal como o próprio termo indica, o *Retrieval-Augmented Generation* descreve uma abordagem em que o modelo procura primeiro informação relevante (*Retrieval*), utiliza essa informação para complementar o contexto (*Augmented*) e, por fim, gera uma resposta com base nesse contexto (*Generation*). A forma como a informação é recuperada pode variar de acordo com os dados disponíveis e com o objetivo da aplicação, sendo possíveis diferentes estratégias:

- **Recuperação por informação estruturada:** utiliza apenas campos selecionados e dados filtrados para encontrar informação relevante.

- **Recuperação semântica:** utiliza *embeddings* (representações de objetos) para encontrar informação com significado semelhante ao conteúdo que está a ser analisado, mesmo quando são utilizadas palavras diferentes.
- **Recuperação híbrida:** combina informação estruturada com pesquisa semântica, permitindo primeiro limitar os resultados com base em determinados campos e depois encontrar os conteúdos mais semelhantes.

2.4 Fine-tuning

O *fine-tuning* consiste em adaptar um modelo de linguagem pré-treinado a uma tarefa ou domínio específico através de um novo treino com dados direcionados. Este processo permite que o modelo aprenda padrões e termos próprios do domínio, tornando as suas respostas mais adequadas ao objetivo pretendido.

Apesar das suas vantagens, o *fine-tuning* requer dados de treino de qualidade e recursos computacionais. Além disso, quando a informação do domínio muda frequentemente, é necessário voltar a treinar o modelo para acompanhar essas alterações. Por isso, em contextos com informação dinâmica, como a gestão de incidentes, o *RAG* pode ser uma abordagem mais adequada para fornecer informação atualizada, enquanto o *fine-tuning* pode ser usado como complemento para melhorar a consistência das respostas (Zhang et al. 2024).

2.5 Métricas de avaliação

A avaliação de uma solução baseada em *LLMs* pode ser feita através de diferentes métricas, uma vez que cada uma permite avaliar um aspeto diferente do sistema. A *Recall-Oriented Understudy for Gisting Evaluation (ROUGE)* permite comparar a resposta gerada com uma resposta de referência, sendo útil para avaliar a similaridade do texto. Para avaliar a qualidade das sugestões de mitigação e dos incidentes recuperados, pode ser utilizada a Precisão no topo K (*Precision@k*), que mede quantos dos primeiros *k* resultados são realmente relevantes (Goel et al. 2024; J. Jiang et al. 2020; Roy et al. 2024; Zhang et al. 2024).

Para além da qualidade dos resultados, é também importante avaliar o custo e o tempo de resposta. O custo por pedido permite medir quanto é necessário gastar, para realizar cada análise através da API, enquanto a latência mede o tempo necessário para obter uma resposta. Desta forma, a avaliação considera não só a qualidade do conteúdo gerado, mas também o custo e o desempenho da solução.

Capítulo 3

Estado da Arte

Este capítulo apresenta o estado da arte sobre o tema em estudo. Para esse efeito, é descrita a metodologia de investigação adotada para analisar a utilização de modelos de linguagem de grande dimensão, que apoia a análise e resolução de incidentes em contextos de operações de TI. De seguida, são apresentados os principais artigos obtidos a partir da revisão da literatura, bem como uma discussão dos aspetos comuns e divergentes identificados nesses estudos. Esta análise permite consolidar uma base de conhecimento que serve de suporte à avaliação da viabilidade do desenvolvimento de uma solução para o problema identificado. Por fim, avaliamos quais são as ferramentas que estão disponíveis no mercado, concluindo quais as funcionalidades que o mercado já cobre.

3.1 Metodologia de Investigação

Optou-se por utilizar um método de pesquisa com base num estudo sistemático de mapeamento ou *Systematic Mapping Study (SMS)*, a fim de realizar a revisão de literatura. Primeiramente, foi definida uma pergunta de investigação, para estudar soluções já existentes no mercado. De seguida, com base na *Research Question (RQ)* definida e com o suporte da metodologia: População, Intervenção, Comparação, Outcome, Contexto e Tipo de estudo (PICOCT), foram identificados os elementos necessários para estruturar a pesquisa. Com a questão já enquadrada, prosseguiu-se para a definição das palavras-chave e dos limites de inclusão e exclusão. Por fim, para obter os artigos, foram utilizadas bases de dados de artigos científicos reconhecidas, tais como, a *ACM Digital Library* e a *IEEE Xplore*.

3.1.1 Pergunta(s) de Investigação

Com base nas dificuldades identificadas pelas equipas *DevOps* e *CorpOps* na análise e resolução de incidentes, surgiu a hipótese de recorrer a ferramentas de Inteligência Artificial (IA), como técnicas de Processamento de Linguagem Natural (PLN), para apoiar a compreensão e o tratamento destes registos. Desta forma, foi formulada a questão: “Como é que as ferramentas de *Large Language Models* podem melhorar a interpretação e análise de registos de incidentes de TI?”.

3.1.2 Metodologia PICOC(T)

A metodologia PICOCT é, geralmente, usada como guia para organizar e clarificar perguntas de investigação, ajudando a estruturar o problema de forma clara. Através desta abordagem, a pergunta é dividida em componentes simples, como a população em estudo, a solução

analisada, os resultados esperados e o contexto, tornando mais fácil de perceber o foco da investigação.

Existem várias variantes desta metodologia, como PICO, PCC ou POT, sendo a escolha da variante dependente do tipo de estudo e da área científica. Neste trabalho, a metodologia PICOCT foi utilizada para apoiar a formulação da pergunta de investigação e para identificar as palavras-chave usadas na pesquisa bibliográfica.

Com base na pergunta de investigação descrita anteriormente, foi possível realizar a decomposição pelos componentes da metodologia e a obtenção de palavras-chave para cada um desses componentes (Tabela 3.1).

Componente	Descrição	Inclusão/Exclusão	Palavras-Chave
P – População	Registos de incidentes	I — Ferramenta ServiceNow E — Estudos sobre gestão de alarmes	"incident management"; "incident analysis"; "incident resolution"; "incident interpretation"; "ticket analysis"; "ticket interpretation"; "ServiceNow"
I – Intervenção	Uso de LLMs para análise de incidentes	I — Técnicas conhecidas de LLMs E — - - -	"Large Language Model"; "LLM"; "Natural Language Processing"; "NLP"; "RAG"; "fine-tuning"
C – Comparação	_____	_____	_____
O – Outcome	Redução do MTTR, melhoria na priorização e diminuição de erros de análise	I — - - - E — Estudos não relacionados com incidentes	"incident"; "ticket"
C – Contexto	Ambientes DevOps/CorpOps	I — Estudos na área E — Estudos fora da área	"DevOps"; "CorpOps"; "IT"
T – Tipo de estudo	Pesquisa empírica ou com base em estudos	I — - - - E — Trabalho teórico ou especulativo	Exclui —> "theoretical"; "speculative"

Tabela 3.1: Metodologia PICOCT

3.1.3 Palavras-Chave

Foi definida uma string de pesquisa (Código 3.1) com o intuito de orientar a revisão bibliográfica. As palavras-chave utilizadas resultaram da aplicação da metodologia PICOCT, garantindo o alinhamento entre a pergunta de investigação e a pesquisa efetuada. Ao longo do processo, a pesquisa assumiu também um caráter exploratório e iterativo, permitindo o

ajuste dos termos sempre que necessário, de acordo com os resultados obtidos e com o foco do trabalho. Os termos foram formulados em inglês, de modo a ampliar o número e a relevância dos estudos encontrados.

```

1      ("incident management" OR "incident analysis" OR "incident
2      resolution" OR "incident interpretation" OR "ticket analysis" OR "
3      ticket interpretation" OR "ServiceNow")
4      AND
5      ("Large Language Model" OR "LLM" OR "Natural Language Processing"
6      OR "NLP" OR "RAG" OR "fine-tuning")
7      AND
8      ("incident" OR "ticket")
9      AND
10     ("DevOps" OR "CorpOps" OR "IT")
11     NOT
12     ("theoretical" OR "speculative")

```

Código 3.1: Termos de Pesquisa

Os termos finais foram posteriormente aplicados com a finalidade de obter os artigos científicos utilizados na investigação.

Os repositórios de pesquisa selecionados, mencionados na Tabela 3.2, possibilitaram a consulta de artigos e procedimentos mais fiáveis, relacionados com a área de estudo.

Repositório	Acesso à plataforma (URL)
ACM Digital Library	https://dl.acm.org/
IEEE Xplore	https://ieeexplore.ieee.org/

Tabela 3.2: Repositórios de Pesquisa

3.1.4 Critérios de Inclusão e Exclusão

Os critérios de inclusão e exclusão adotados neste trabalho, e definidos nas Tabelas 3.3 e 3.4, são adicionais ao já estipulados na metodologia PICOCT e tiveram como principal objetivo delimitar o âmbito da revisão da literatura e garantir a sua relevância para a questão de investigação. São filtros adicionais que não podem fazer parte dos termos de pesquisa mas podem, normalmente, ser configurados nas próprias configurações dos repositórios de pesquisa.

Devido ao elevado número de artigos foi definido no critério *C11* que, dos procedimentos originários de conferências, apenas seriam considerados trabalhos resultantes da *International Conference on Software Engineering (ICSE)*, da *ACM International Conference on the Foundations of Software Engineering (FSE)* e a *Automated Software Engineering (ASE)*.

A *ICSE* é uma das principais conferências internacionais, focando-se em inovações e tendências da área. Já a *FSE*, também conhecida como *ESEC/FSE*, destaca-se pelo foco em métodos, ferramentas e técnicas para o desenvolvimento e manutenção de *software*. Por sua vez, a *ASE* centra-se na automação de processos de engenharia de *software*, e em particular nos estudos que envolvem o uso de técnicas de IA no apoio a atividades operacionais, como é o caso deste estudo.

Identificador	Critério de Inclusão
CI1	Artigos provenientes de conferências reconhecidas e relacionadas com o tema

Tabela 3.3: Critérios de Inclusão

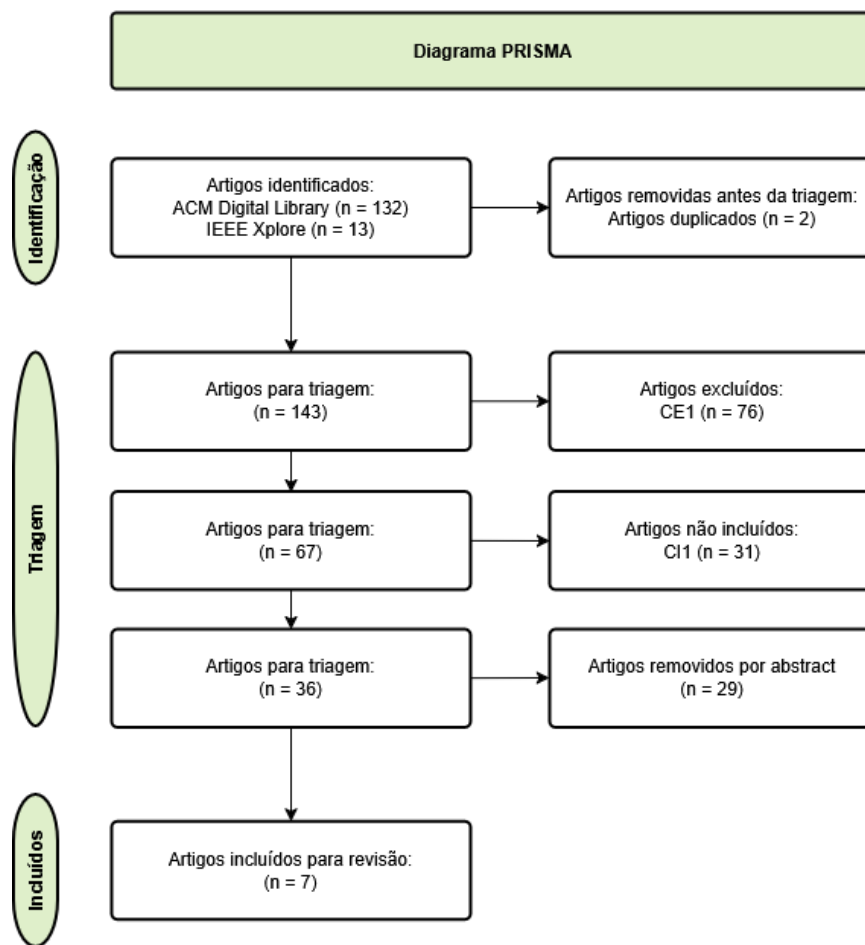
Com o critério *CE1* procurou-se dar prioridade a trabalhos mais recentes, reduzindo a probabilidade de incluir tecnologias que já não são relevantes.

Identificador	Critério de Exclusão
CE1	Publicado antes de janeiro de 2020
CE2	Artigos que não contenham resumo ou <i>abstract</i>
CE3	Artigos duplicados ou com conteúdos redundantes

Tabela 3.4: Critérios de Exclusão

3.1.5 PRISMA

Para organizar a revisão de literatura, recorreu-se ao método *Preferred Reporting Items for Systematic reviews and Meta-Analyses (PRISMA)*, que ajuda a tornar o processo de seleção de estudos **mais claro e transparente**. Com base nos artigos obtidos nos repositórios de pesquisa, e filtrando esses resultados pelos critérios de inclusão e exclusão, obtivemos os artigos que foram usados como base para esta investigação.

Figura 3.1: Diagrama *PRISMA*

Como demonstrado na Figura 3.1, foram obtidos 145 artigos dos dois repositórios de pesquisa dos quais 2 eram duplicados. Após a identificação inicial, os artigos foram filtrados pelos critérios de exclusão, onde 76 dos mesmos foram descartados por terem sido publicados antes de janeiro de 2020. De seguida, procedeu-se à filtragem pelos critérios de inclusão, que removeu 31 artigos que não se enquadravam nas conferências selecionadas. Por fim, 29 artigos foram excluídos, após leitura do *abstract*, por não se enquadrarem no tema. Sobraram, dessa forma, 7 artigos para analisar na revisão de literatura.

3.2 Revisão de Literatura

Nesta secção, é demonstrada a análise dos resultados obtidos pela metodologia *PRISMA*, abordando os temas por categorias relevantes a esta investigação.

3.2.1 Análise de Causa Raiz e Correlação de Incidentes

As abordagens atuais para identificar as causas dos incidentes são, ainda, muito reativas e manuais, sendo que toda a investigação da origem do problema depende, quase exclusivamente, da experiência dos *OCEs* e da consulta da documentação técnica e registos das aplicações. Enquanto funcional, para ambientes controlados, estas abordagens tendem a apresentar limitações quando aplicadas a sistemas de grande escala.

Em paralelo, nos ambientes de grande escala, um único problema pode propagar-se por vários serviços e originar múltiplas ocorrências aparentemente independentes, fenômeno já identificado em sistemas reais de gestão de incidentes (Yujun Chen et al. 2020). Também afirmado por Yiru Chen et al. (2024), em sistemas de grande escala, uma única falha pode acionar vários alertas, uma vez que os serviços são altamente interligados e dependem uns dos outros. Consequentemente, é gerado um grande volume de alertas e incidentes em pouco tempo, o que dificulta a identificação dos alertas pertencentes à mesma falha para análise da causa raiz. Esta recorrência e repetição de incidentes está, assim, ligada ao aumento do tempo despendido na procura da causa e do melhor processo de mitigação e conduz, frequentemente, a diagnósticos redundantes ou demorados.

Torna-se importante identificar correlações entre incidentes que, à primeira vista, pareçam diferentes. Esta ligação entre ocorrências pode ter a mesma origem, os mesmos sintomas ou até descrições semelhantes. Trabalhos mais recentes mostram que a identificação de relações entre incidentes ou alertas relacionados é fundamental para apoiar atividades como a análise de causa raiz e a mitigação eficiente (J. Jiang et al. 2020; Roy et al. 2024).

Nos últimos anos, foi evidente o aumento da ênfase na reutilização do histórico de incidentes e na identificação dos padrões recorrentes entre eles. Estudos como J. Jiang et al. (2020) mostraram que a maioria dos incidentes acontecem de forma repetida ao longo do tempo e, muitas vezes, são mitigados utilizando o mesmo procedimento. Este trabalho apresenta o conceito de recomendação automática por meio de *Troubleshooting Guide (TSG)s*, demonstrando que a avaliação de incidentes anteriores pode ajudar eficientemente o processo de solução, diminuindo o esforço cognitivo das equipas de operações. De acordo com o mesmo estudo, apenas 27.2% dos incidentes tinham *TSGs* associados, revelando falta de documentação e, principalmente, que em maioria dos incidentes, os membros da equipa de suporte têm necessidade de investigar a fundo a causa das ocorrências. De notar ainda que, o artigo afirma que, 36.2% dos incidentes ocorreram pelo menos 2 vezes, podendo levar à reutilização dos métodos de mitigação.

Neste sentido, abordagens baseadas em aprendizagem automática e, mais recentemente, em modelos de linguagem de grande escala, têm vindo a ser exploradas para consolidar informação dispersa e apoiar a tomada de decisão durante incidentes complexos (Goel et al. 2024; Y. Jiang et al. 2024), reforçando a importância da correlação de *tickets* como etapa central na gestão operacional moderna.

3.2.2 Sugestões de Mitigação e Uso de Incidentes Anteriores

A maioria dos incidentes que surgem não são totalmente novos, mas sim recorrências de situações já observadas no passado. Ainda assim, identificar rapidamente estes casos de semelhança e as respetivas ações de mitigação continua a ser um desafio para os membros das equipas de suporte. O trabalho de J. Jiang et al. (2020) mostra, uma vez mais, que apenas de 27.2% dos incidentes possuem um guia de mitigação associado e que os *OCEs* despendem 36.3% do tempo total de mitigação apenas à procura do *TSG* correto. No mesmo estudo, a abordagem proposta foi capaz de recomendar o guia mais adequado como primeira sugestão em 80.3% dos casos, confirmando, assim, o impacto direto do reaproveitamento de conhecimento histórico na redução do tempo de resposta.

Trabalhos mais recentes, exploram o uso de modelos de linguagem de grande escala para automatizar este processo de investigação de uma forma mais flexível. O estudo de Roy et al. (2024) demonstra que agentes baseados em *LLM* conseguem gerar sugestões de mitigação

quando combinam descrições de incidentes com informação histórica e dados adicionais, como métricas e registos das aplicações, o que faz melhorar a qualidade das sugestões. De igual modo, o artigo de Goel et al. (2024) evidencia que a inclusão de contexto proveniente de diferentes fases do ciclo de vida do software melhora o desempenho dos modelos na geração de recomendações, quando comparados com abordagens que utilizam apenas uma fonte de informação.

Estes resultados, indicam que a análise de incidentes anteriores, quando suportada por fontes de informação adicionais, pode reduzir a carga cognitiva dos membros de suporte e tornar o processo de mitigação mais rápido, mais consistente e mais independentemente do nível de experiência do elemento de prevenção.

3.2.3 Apoio à Contextualização de Incidentes com LLMs

Para além dos temas abordados anteriormente, existe ainda um desafio inicial que os membros das equipas de suporte têm de superar. Antes de atuar sobre um incidente, seja por análise própria, por sugestões de mitigação ou até *TSGs*, os *OCEs* devem perceber o incidente em si, analisando as informações que são dadas de forma automática ou, quando criados manualmente, pelos clientes. Essas informações são muitas vezes pouco estruturadas e ambíguas.

Esta falta de contexto na fase inicial, impele os *OCEs* a iniciar uma investigação com base em informação limitada, o que faz com que os elementos de prevenção realizem várias tentativas para entender o impacto real do incidente e os serviços que são afetados. Em equipas com aplicações de grande escala, onde múltiplos incidentes podem ocorrer em simultâneo, uma primeira análise incompleta do incidente pode contribuir para atrasos na resposta e aumentar a pressão sobre os *OCEs*.

Desta forma, resolver incidentes torna-se num processo dependente da experiência dos membros das equipas de suporte, o que realça a necessidade da exploração de diferentes fontes de informação, como a análise de incidentes anteriores, métricas ou registos de eventos dos sistemas. Como demonstrado por Roy et al. (2024), agentes baseados em *LLMs* conseguem integrar várias fontes de dados durante a investigação, o que permite uma compreensão mais rápida do problema à medida que nova informação é incorporada.

Goel et al. (2024) mostra ainda que, incluir contexto adicional proveniente de diferentes fases do ciclo de vida do software, pode melhorar a capacidade dos *LLMs* em apoiar tarefas de análise e investigação. Este enriquecimento contextual, contribui para reduzir a incerteza inicial associada aos incidentes e orientar os *OCEs* de forma mais eficaz no primeiro contacto com os *tickets*. Adicionalmente, Y. Jiang et al. (2024) também apontam que os *LLMs* podem facilitar o acesso à informação operacional relevante, tornando tarefas que exigiam conhecimento específico sobre ferramentas e linguagens internas em tarefas automatizadas.

Em suma, a consolidação de informação histórica e contextual dos incidentes, apoiada por modelos de linguagem de grande escala e fontes de informação adicionais, desempenha um papel fundamental na redução da incerteza associada à investigação inicial de incidentes.

3.2.4 Técnicas e Modelos para Análise de Incidentes

As principais técnicas identificadas na literatura são:

- **RAG:** Combinação de mecanismos de recuperação de informação com modelos de linguagem para gerar respostas com mais contexto. Foi utilizado na geração de *queries* e apoio à análise de incidentes, apresentando melhorias na qualidade das respostas quando comparado com *LLMs* sem mecanismos de *retrieval*. Como limitação, depende fortemente da qualidade dos dados recuperados. (Goel et al. 2024; Y. Jiang et al. 2024)
- **Agentes de LLM:** Agentes baseados em modelos de linguagem capazes de executar raciocínio com várias etapas e interagir com ferramentas externas. São particularmente eficazes na análise de causa raiz e apresentam maior precisão do que abordagens baseadas em *LLM* simples. No entanto, implicam maior custo computacional e tempo de resposta. (Roy et al. 2024)
- **Chain-of-Thought (CoT):** Técnica de instruções que incentiva o modelo a gerar raciocínio passo-a-passo. Melhora a interpretabilidade e a qualidade das respostas em tarefas de *debugging* e análise de incidentes, embora aumente o tempo de análise. (Y. Jiang et al. 2024; Roy et al. 2024)
- **Clustering:** Permite agrupar incidentes com base na sua similaridade, sendo útil na correlação de incidentes e apresentando uma maior robustez do que heurísticas simples, mas sendo sensível à escolha de parâmetros. Um exemplo de um algoritmo referido na revisão de literatura, foi o *Density-Based Spatial Clustering of Applications with Noise (DBSCAN)*. (Yujun Chen et al. 2020)
- **Graph Neural Networks (GNNs):** Modelos que representam sistemas como grafos, capturando, assim, relações entre componentes. Estes modelos, têm sido propostas para prever relações entre alertas, sendo avaliadas através de métricas como *Area Under the Curve (AUC)* e *F1-Score*, demonstrando desempenho superior face a métodos tradicionais. Contudo, possuem elevada complexidade computacional. (Yiru Chen et al. 2024)
- **Modelos de Grafos Temporais:** Extensões de *GNNs* que incorporam informação temporal, e que permitem modelar a evolução de incidentes ao longo do tempo. Oferecem melhor desempenho do que grafos estáticos, no entanto, são mais complexos de implementar. (Yiru Chen et al. 2024)
- **Learning-to-Rank:** Abordagem que permite ordenar resultados com base na sua relevância. É utilizada na recomendação de ações de mitigação, superando métodos heurísticos, embora necessite de dados anotados para treino. (J. Jiang et al. 2020)

3.2.5 Conclusão

Com este capítulo, foi possível verificar, com base nos trabalhos analisados, que a gestão de incidentes em sistemas de grande escala é, ainda, muito afetada pela fragmentação da informação relacionada com as falhas que causam os incidentes, pela recorrência e duplicação de *tickets* em curtos intervalos de tempo e pela dependência da experiência dos *OCEs*. Assim, a literatura, mostrou que o uso de IA, como *LLMs* ou *GNNs*, facilita a contextualização inicial dos incidentes aos membros da equipa de suporte e ajuda na mitigação das falhas ao fornecer detalhes e sugestões de resolução com base em registos de incidentes anteriores.

Em síntese, e respondendo à pergunta inicial "Como é que as ferramentas de *Large Language Models* podem melhorar a interpretação e análise de registos de incidentes de TI?", é possível afirmar que estes Modelos de Linguagem de Grande Escala têm um papel importante no

apoio da compreensão inicial, na análise e na procura de formas de mitigar os incidentes. São, assim, um apoio no desempenho por parte do trabalho dos membros das equipas de prevenção.

No entanto, a análise crítica da literatura, mostrou que os estudos efetuados foram realizados com conjuntos de dados (*datasets*) estáticos e abordam apenas fases específicas do ciclo de vida de um incidente. Assim, o desenvolvimento de uma nova solução, que agrega várias fases deste ciclo e use um conjunto de dados dinâmicos, pode acarretar riscos ainda não identificados por estudos já realizados.

No capítulo 4, será discutido o futuro desta investigação, apresentando uma proposta de implementação capaz de alcançar os objetivos propostos no capítulo 1.

3.3 Soluções no Mercado

A Inteligência Artificial é um tema em alta em todos os mercados e áreas, sendo esta uma área que não escapa à automatização de processos. Dessa forma, o mercado já apresenta algumas soluções para o correlacionamento de incidentes, para o resumo dos mesmos, para sugestões de mitigação e para análise de informação adicional (eventos das aplicações). Nesta secção abordamos quatro das principais ferramentas existentes, abordando as suas funcionalidades fundamentais.

3.3.1 AWS AIOps

A AWS recorreu a técnicas de IA e *Machine Learning* (ML), de forma a "ajudar a aprimorar, acelerar e automatizar os processos de operações em nuvem" (*Operações de IA* 2026). O *AWS AIOps*, permite que os *OCEs* observem facilmente a sua carga de trabalho, acelerando a solução de problemas operacionais e a toma de medidas para resolver e mitigar problemas, melhorando o *TTM*.

As principais funcionalidades da sua ferramenta, são:

- **Encontrar a causa raiz de um problema:** É possível configurar o ambiente para iniciar uma investigação mal um alarme seja disparado;
- **Sugestões de correção:** A AWS fornece guias do seu próprio ambiente para a resolução de problemas relacionados com ferramentas AWS;
- **Capacitar operadores sem experiência:** A ferramenta analisa pontos de dados para descobrir relações entre serviços e desenvolver uma compreensão de como eles funcionam juntos;
- **Consulta de dados de telemetria usando linguagem natural:** A AWS permite escrever perguntas em linguagem natural sem precisar de aprender linguagens de consulta complexas.

3.3.2 New Relic AIOps

A plataforma de *AIOps* da *New Relic* oferece funcionalidades orientadas por IA e ML, utilizando técnicas como RAG, para gerar interfaces que ajudam as equipas a resolver incidentes com mais rapidez (*AIOps and Applied Intelligence | New Relic* 2026).

As funcionalidade relevantes da plataforma, são:

- **Deteção de anomalias instantânea:** Identifica problemas emergentes ou desvios do sistema usando modelos adaptativos para conectar sintomas a todos os níveis da arquitetura;
- **Correlação de alertas:** Identifica eventos de alerta semelhantes numa única questão para facilitar a resolução de problemas;
- **Priorização e conexão com outros serviços:** Oferece a possibilidade de configurar alarmes como mais prioritários e integrar sistemas de notificação, como SMS ou *Microsoft Teams*.

3.3.3 Atlassian Ops Guide Agent

O agente criado pela *Atlassian (Ops Guide)*, é um agente projetado para ajudar na gestão de alertas e incidentes com mais eficiência, fornecendo contexto histórico e recomendando ações, simplificando assim as tarefas das equipas de suporte (*Using the Ops Guide agent / Rovo 2026*).

Este agente fornece a possibilidade de:

- **Pesquisa em Linguagem Natural:** Permite pesquisar por alertas e incidentes em linguagem natural, convertendo de forma automática em sintaxe de pesquisa da ferramenta;
- **Correlação de incidentes:** O agente, analisa o histórico de incidentes anteriores para contextualizar com informação adicional os *tickets*;
- **Criação de Post-Mortems:** Através da sua ferramenta *Confluence*, o agente concede a opção de criar revisões pós incidentes baseado nos detalhes de um *ticket*.

3.3.4 Elastic AI Assistant

O assistente de Inteligência Artificial fornecido pela *Elastic*, foge um pouco ao âmbito das ferramentas apresentadas anteriormente, uma vez que este serviço não gere incidentes mas é sim uma ferramenta de observabilidade, onde é possível visualizar o registo dos eventos de serviços. Assim, este assistente é uma integração com *LLMs* que ajuda a perceber, analisar e interagir com os dados armazenados (*Elastic AI Assistant for Observability and Search / Elastic Docs 2026*).

Esta plataforma, ajuda a:

- **Descodificar mensagens de erro:** Interpreta rastros dos serviços e registos de erros para identificar as causas principais de falhas;
- **Identificar estrangulamentos de desempenho:** Encontra operações que consomem muitos recursos e consultas lentas na ferramenta;
- **Criar consultas em Linguagem Natural:** A partir de linguagem natural, converte sintaxe *Domain-Specific Language (DSL)* da consulta para linguagem de consultas do *ElasticSearch* e executa-as diretamente a partir da interface de conversa.

3.3.5 Conclusão

Através das tabelas 3.5 e 3.6 conseguimos distinguir as capacidades oferecidas pelas soluções no mercado e as técnicas utilizadas para as implementar.

É possível que soluções mais tradicionais, como *AWS AIOps* e *New Relic*, têm maior foco em capacidades de correlação e análise de causa raiz suportadas por técnicas de *machine learning* e análise de dados, apresentando um suporte limitado para contextualização baseada em modelos de linguagem e focando-se apenas no seu ambiente de ferramentas.

Por outro lado, soluções mais recentes, como o *Atlassian Ops Guide Agent* e o *Elastic AI Assistant*, recorrem a modelos de linguagem, agentes e técnicas de *RAG*, permitindo melhorar significativamente a contextualização e o suporte à decisão.

Esta distinção evidencia uma evolução das abordagens, passando de sistemas centrados na análise de dados para sistemas orientados à interpretação, contextualização e assistência inteligente onde se torna evidente o uso mais frequente de agentes *LLM* e sistemas baseados em *RAG*.

Solução	Correlação	Causa Raiz	Mitigação	Contextualização
AWS AIOps	Sim	Sim	Sim	Parcial
New Relic AIOps	Sim	Sim	Não	Não
Atlassian Ops Guide Agent	Sim	Sim	Não	Sim
Elastic AI Assistant	Não	Sim	Não	Sim

Tabela 3.5: Funcionalidades de Soluções Existentes

Solução	Agentes LLM	RAG	Clustering	Grafos
AWS AIOps	Não	Não	Sim	Não
New Relic AIOps	Não	Não	Sim	Sim
Atlassian Ops Guide Agent	Sim	Sim	Não	Não
Elastic AI Assistant	Sim	Sim	Não	Não

Tabela 3.6: Técnicas usadas por Soluções Existentes

Capítulo 4

Análise

Após a revisão da literatura, realizada no capítulo 3, conseguimos chegar à conclusão que as ferramentas de *LLM* quando utilizadas para correlação de incidentes, análise do histórico de *tickets* e sugestões de mitigação, podem ajudar os membros das equipas de suporte a compreender melhor os incidentes e resolvê-los de uma forma mais eficaz e eficiente.

Assim, a solução, através da integração com um *LLM*, deve ser capaz de apoiar os *OCEs* na interpretação, análise e mitigação dos incidentes. Esta solução é baseada, numa fase inicial, pela ligação entre o incidente a resolver e os registos anteriores, aumentando a informação do incidente atual numa linguagem perceptível para qualquer membro da equipa de suporte, com mais ou menos experiência, e em qualquer contexto de trabalho e pressão. Com o apoio de integrações externas, como *Confluence*, pretende-se também aumentar o contexto do serviço afetado de forma a indicar uma mitigação mais viável. Com a correlação de incidentes funcional e documentação dos próprios produtos, procura-se encontrar sugestões de mitigação para o utilizador que permitam uma rápida ação para resolver, temporariamente ou definitivamente, o problema, diminuindo o TMR.

4.1 Desenho da Solução

Como referido anteriormente, o objetivo da solução passa por integrar com diversos serviços externos, tais como:

- **Serviço de incidentes:** apresenta-se como sendo a base da solução, é a partir deste serviço que retiramos a informação do incidente a ser analisado e do histórico de incidentes, para realizar a correlação;
- **Serviço de documentação:** com informação retirada do incidente, fornece documentação do serviço afetado;
- **Motor LLM:** outro elemento que devemos considerar como base da solução, visto que é responsável por fazer a ligação entre o incidente principal, os incidentes relacionados e a documentação do serviço devolvendo um resumo em linguagem natural e as sugestões de mitigação priorizadas.

Assim, a solução proposta combina a informação obtida do serviço de incidentes e documentação com as capacidades de análise do motor *LLM*. Esta combinação permite analisar o incidente atual com base no histórico disponível, gerar uma descrição em linguagem natural e apresentar sugestões de mitigação.

4.2 Dataset

Um *dataset* é uma coleção estruturada de dados organizados e armazenados juntos para realizar alguma análise ou processamento. Os dados presentes num dataset são normalmente relacionados entre si. Neste caso, o dataset trata-se de um conjunto volumoso de incidentes no foro de várias equipas num projeto comum.

A gestão do dataset é realizada por um serviço externo (*ServiceNow*) ao qual o acesso é feito através de uma Interface de Programação de Aplicações (*API*). De forma a manter o conjunto de dados o mais recente e mais atualizado possível, é utilizada a API em cada pedido realizado ao serviço em desenvolvimento.

4.2.1 Estrutura

Para a construção do conjunto de dados, foram considerados todos os campos do incidente principal (excluindo os dados presentes na tabela 4.2). Já os incidentes relacionados mantiveram apenas os campos que contribuíam diretamente para a compreensão da falha e para a geração das recomendações (tabela 4.1). Os restantes campos foram descartados porque não acrescentavam valor operativo relevante ou introduziam ruído excessivo ao processo de análise.

Campo	Utilização no estudo
<i>incident ID</i>	Identifica unicamente cada incidente e permite fazer a correlação com registos relacionados.
<i>short description</i>	Permite resumir o problema e identificar incidentes com títulos semelhantes.
<i>description</i>	Fornece o contexto detalhado da falha, sintomas e impacto operacional.
<i>state</i>	Ajuda a distinguir incidentes ativos de incidentes fechados, resolvidos ou cancelados.
<i>assignment group</i>	Permite identificar a equipa responsável pelo incidente.
<i>resolved_at</i>	Utilizado para ordenar incidentes por relevância temporal e maior proximidade no tempo.
<i>closed_notes</i>	Campo com as notas de resolução do incidente.
<i>comments</i>	Fornece evidências operacionais, passos de diagnóstico e resoluções anteriores.
<i>hold_reason</i>	Campo utilizado quando um incidente aguarda resposta do autor.

Tabela 4.1: Campos Utilizados do Dataset

Campo	Motivo de exclusão
<i>caller_id</i>	Autor do incidente não é relevante para a análise técnica do registo e pode introduzir dados sensíveis.
<i>assigned_to</i>	Analista do incidente não é relevante para a análise técnica do registo e pode introduzir dados sensíveis.
<i>resolved_by</i>	Analista que resolveu o incidente não é relevante para a análise técnica do registo e pode introduzir dados sensíveis.
<i>attachments</i>	A sua utilização no início do projeto não é relevante devido à natureza da maioria do conjunto de dados mas está considerado para trabalho futuro.

Tabela 4.2: Campos Descartados do Dataset

4.2.2 Origem dos Incidentes

Os incidentes podem surgir principalmente de duas formas: manualmente, quando são criados por equipas de suporte após ser detetado um problema, ou automaticamente, quando são gerados por sistemas de monitorização. Esta diferença é importante porque a forma como o incidente é criado pode influenciar a informação disponível no início da análise, a qualidade da sua descrição e o tempo necessário para começar a resolver o problema.

Nos incidentes gerados automaticamente, a informação tende a ser mais técnica e estar relacionada com o funcionamento dos sistemas, como falhas de serviços, métricas fora do normal ou problemas com dependências. Por outro lado, os incidentes criados manualmente podem ter mais informação sobre o problema sentido pelo utilizador e o seu impacto, mas a descrição pode variar bastante e nem sempre conter todos os dados necessários para iniciar a análise.

Por este motivo, perceber a origem do incidente é importante para a sua análise inicial, para identificar o problema, determinar a equipa responsável e perceber se é necessário obter informação adicional antes de avançar para a resolução.

Na imagem 4.1, é possível ver que a maioria dos incidentes recebidos são de origem automática (Alert) e que existe uma quantidade limitada de incidentes manuais (Standard-UI, Portal e Email).

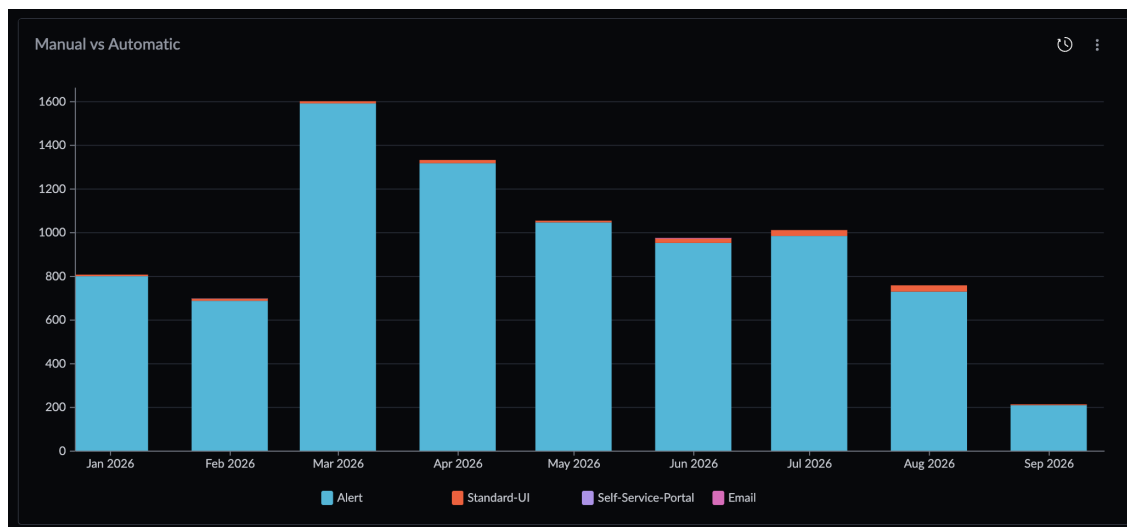


Figura 4.1: Incidentes Manuais vs Automáticos

É ainda possível, confirmar na imagem 4.2 a quantidade e a percentagem exata de incidentes automáticos, 8306 incidentes que corresponde a 98.41% do dataset, e de incidentes manuais, 134 incidentes que corresponde a 1.59% do conjunto de dados.

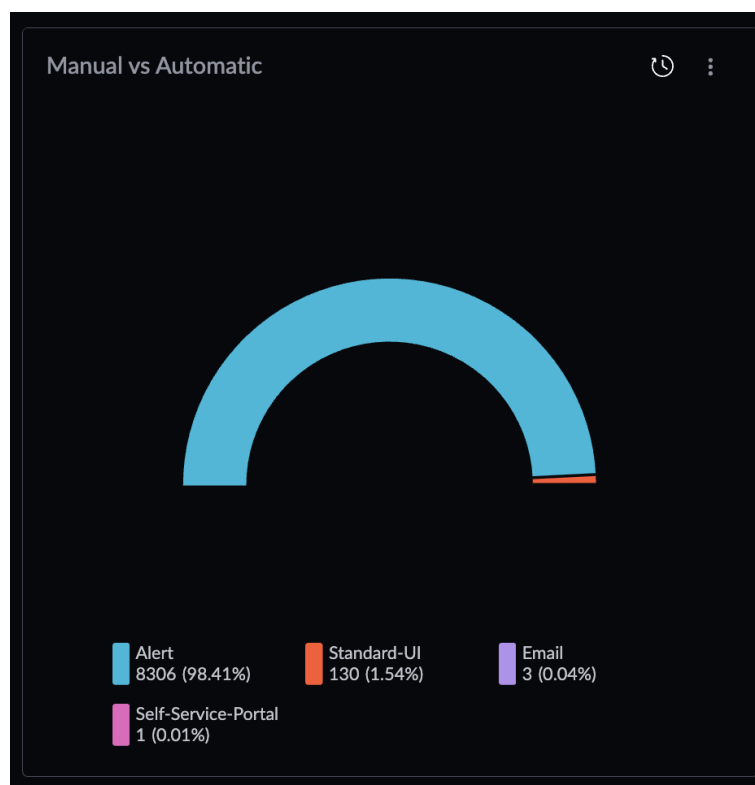


Figura 4.2: Percentagem de Incidentes Manuais vs Automáticos

Podemos assim concluir, que a maioria de dados presentes no *dataset* correspondem a incidentes automáticos que por sua vez têm menos complexidade lexical, uma vez que, para um mesmo problema são gerados com um título exatamente similar.

4.2.3 Destino dos Incidentes

Após um incidente ser criado, pode continuar com a equipa que iniciou a análise ou ser encaminhado para outra equipa, dependendo do tipo de problema ou do serviço afetado. Em ambientes com várias equipas, é comum que a equipa que recebe inicialmente o incidente não tenha o conhecimento necessário para o resolver, sendo necessário direcioná-lo para a equipa mais adequada. A atribuição inicial é muitas vezes feita com base no serviço ou na área de negócio afetada, o que nem sempre corresponde à origem técnica do problema.

A encaminhamento de incidentes pode ter impacto direto no tempo necessário para a sua resolução. Quando um incidente é encaminhado para a equipa errada, pode haver perda de tempo, repetição de trabalho e atrasos na investigação, podendo levar a um maior impacto em incidentes críticos. Além disso, decidir para que equipa deve ser encaminhado pode exigir uma análise mais detalhada do incidente e a consulta de casos anteriores. Por isso, mecanismos que ajudem a classificar os incidentes, encontrar casos semelhantes e identificar a equipa mais adequada podem facilitar este processo e reduzir o tempo de resolução.

A análise do ano corrente, presente na imagem 4.3, demonstra que a maioria dos incidentes não são redirecionados, e que existe uma quantidade diminuta de incidentes reencaminhados uma ou mais vezes.

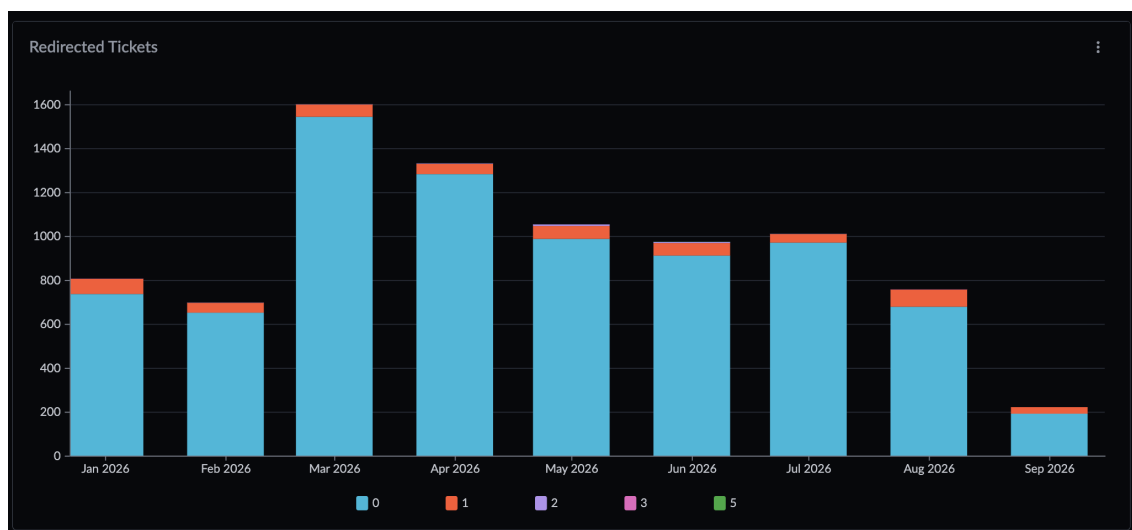


Figura 4.3: Incidentes Redirecionados

É ainda possível, confirmar na imagem 4.4 a quantidade e a percentagem exata de incidentes não encaminhados, 7949 incidentes que corresponde a 94.08% do dataset, de incidentes redirecionados apenas uma vez, 480 incidentes que corresponde a 5.68% do conjunto de dados e os que são reencaminhados mais do que uma vez, 20 incidentes correspondentes à quantia restante.

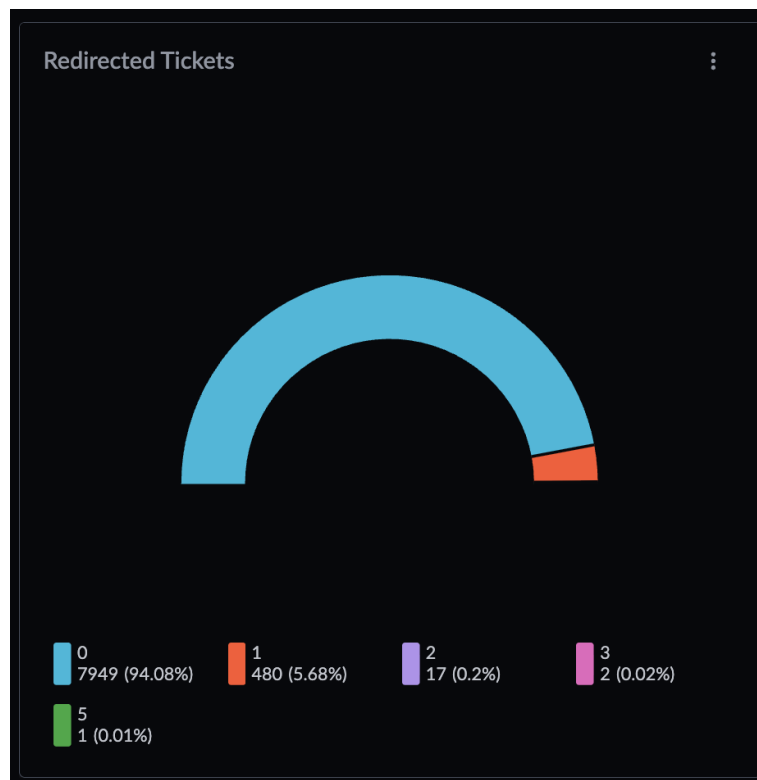


Figura 4.4: Percentagem de Incidentes Redirecionados

Incidentes reencaminhados duas ou mais vezes, podem indicar maus processos de análise e reencaminhamento, o que pode, novamente, atrasar na resolução de problemas no serviço.

No contexto do produto a ser analisado, a maioria de incidentes são redirecionados 0 vezes, o que indica que foram atribuídos corretamente, porém este número pode ser enganador, visto que, normalmente, incidentes automáticos são corretamente encaminhados. Isto significa assim, que apesar do valor de incidentes reencaminhado parecer escasso, analisando apenas o conjunto de registros manuais, estes valores são completamente diferentes, como mostra a imagem 4.5.

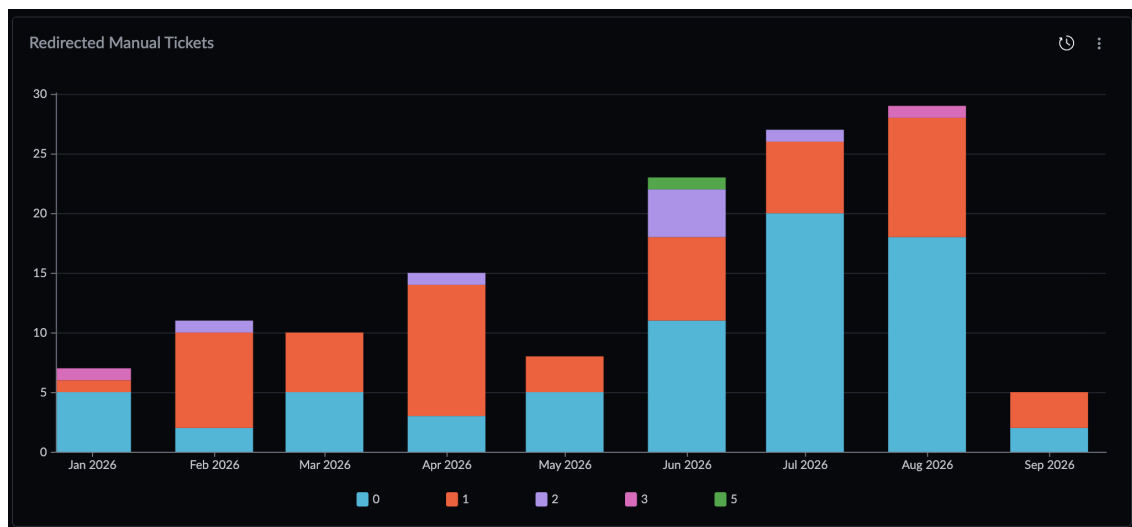


Figura 4.5: Incidentes Manuais Redirecionados

Podemos então confirmar, na imagem 4.6, que a quantidade de incidentes manuais que são redirecionados é de 47.41%, sendo assim uma amostra mais significativa.

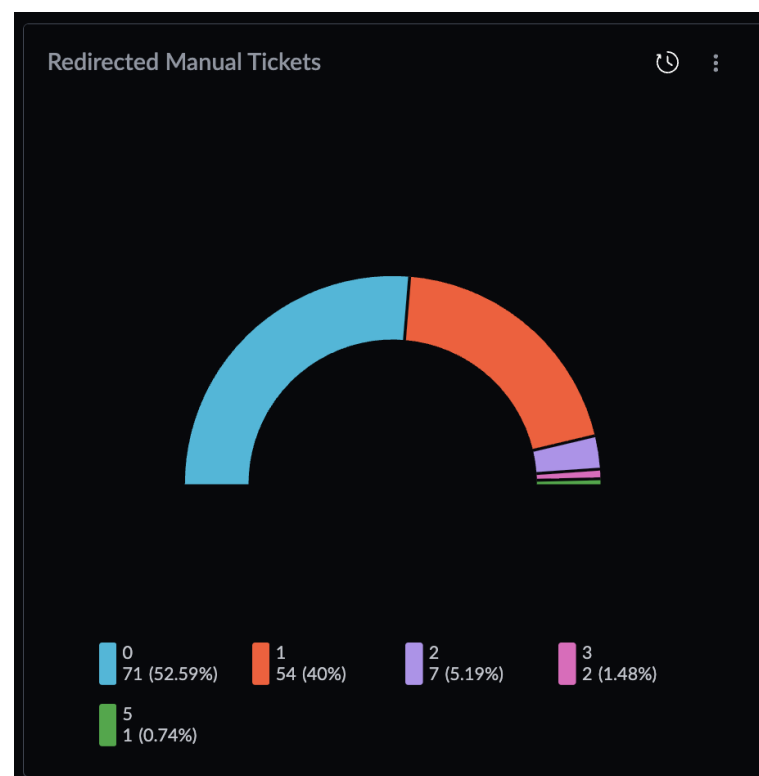


Figura 4.6: Percentagem de Incidentes Manuais Redirecionados

Deduz-se portanto, que é importante realizar uma boa análise do incidente, com recurso a histórico e incidentes relacionados, com o objetivo de indicar ao utilizador se o mesmo deve mitigar o problema ou encaminha-lo para a equipa mais competente.

4.2.4 Conclusão

A análise do *dataset* permite concluir que o foco inicial da solução deve estar nos incidentes gerados automaticamente, uma vez que representam 98,41% dos registos analisados. Estes incidentes apresentam normalmente informação mais estruturada e padrões semelhantes, tornando possível utilizar o histórico para identificar problemas recorrentes e apoiar a sua resolução.

Apesar de representarem uma parcela menor do conjunto de dados, os incidentes manuais apresentam maior variabilidade e uma maior necessidade de análise, sendo também mais frequentemente reencaminhados entre equipas. Por este motivo, a solução deverá inicialmente procurar apoiar a resolução dos incidentes automáticos, sem deixar de considerar os incidentes manuais como um caso importante para a análise e evolução futura da solução.

4.3 Engenharia de Requisitos

A engenharia de requisitos é uma etapa importante no desenvolvimento de software, uma vez que permite identificar e definir as principais funcionalidades do serviço e as condições que deve cumprir. Nesta fase são recolhidas e organizadas as necessidades do sistema, servindo posteriormente como base para o seu desenvolvimento e avaliação. Os requisitos podem ser divididos principalmente em requisitos funcionais e não funcionais.

4.3.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve disponibilizar e as ações que deve ser capaz de realizar. No contexto deste projeto, os requisitos estão relacionados principalmente com a receção e análise de incidentes, a procura de incidentes semelhantes, a utilização dessa informação pelo *LLM* e a geração e priorização de sugestões de mitigação, visíveis na tabela 4.3.

Para definir a prioridade dos requisitos, foi utilizado o método *MoSCoW*, que permite dividir os requisitos em quatro níveis de prioridade:

- **Must have** - Essencial para o funcionamento da solução;
- **Should have** - Importante mas não impede o funcionamento da solução caso não seja implementado;
- **Could have** - Pode ser incluído caso existam recursos e tempo disponíveis;
- **Won't have** - Não está nos planos imediatos do projeto, mas pode fazer parte dos planos futuros.

Esta classificação permite definir quais as funcionalidades que devem ser desenvolvidas primeiro e ajuda a manter o foco nos objetivos principais da solução.

Identificador	Descrição	Prioridade
RF-01	O sistema deve permitir receber um incidente para análise através do serviço de incidentes	Must have
RF-02	O sistema deve obter os dados relevantes do incidente, incluindo a descrição, serviço afetado, severidade e outros campos disponíveis	Must have
RF-03	O sistema deve procurar no histórico por incidentes relacionados com o incidente recebido	Must have
RF-04	O sistema deve utilizar os incidentes relacionados como contexto para a análise realizada pelo <i>LLM</i>	Must have
RF-05	O sistema deve gerar um resumo do incidente em linguagem natural	Must have
RF-06	O sistema deve gerar sugestões de mitigação com base na informação disponível e nos incidentes relacionados	Must have
RF-07	O sistema deve ordenar as sugestões de mitigação de acordo com a sua relevância	Must have
RF-08	O sistema deve apresentar a informação utilizada para suportar as sugestões geradas, permitindo consultar os incidentes relacionados	Must have
RF-09	O sistema deve permitir identificar a equipa mais adequada para tratar o incidente, quando essa informação fizer parte do objetivo da solução	Should have
RF-10	O sistema deve permitir configurar a utilização, ou não, de comentários e notas de análise dos incidentes	Should have
RF-11	O sistema deve permitir configurar o modelo utilizado pela <i>LLM</i>	Could have
RF-12	O sistema deve procurar informação relevante na documentação do serviço afetado, resumir o conteúdo encontrado e utilizá-lo na análise do incidente	Could have
RF-13	O sistema deve consultar os logs do serviço afetado para encontrar informação relacionada com o incidente e utilizá-la na análise	Won't have

Tabela 4.3: Requisitos Funcionais

4.3.2 Requisitos Não Funcionais

Os requisitos não funcionais definem características que o sistema deve cumprir, mas que não estão diretamente relacionadas com uma funcionalidade específica. São, normalmente, considerados aspetos como o desempenho, a segurança, a fiabilidade e manutenção da solução, tal como os definidos na tabela 4.4. Estes requisitos são importantes para garantir que o sistema não só funciona corretamente, mas que também pode ser utilizado de forma adequada num ambiente real.

Identificador	Descrição
<i>RNF-01</i>	O sistema não deve expor informação confidencial dos incidentes a utilizadores ou serviços que não tenham autorização para a consultar
<i>RNF-02</i>	O sistema deve estar organizado de forma a permitir adicionar, substituir ou remover serviços externos sem impacto no restante serviço
<i>RNF-03</i>	O sistema deve limitar a quantidade de informação enviada para o <i>LLM</i> , de forma a manter o custo de cada análise dentro de valores aceitáveis

Tabela 4.4: Requisitos não Funcionais

4.4 Conclusão

Neste capítulo pudemos observar o planeamento da solução desenvolvida, sendo apresentadas as funcionalidades principais e as integrações externas. Foram ainda abordadas, estatísticas sobre o conjunto de dados que permitem perceber qual a maior área de impacto do serviço. Por fim, foram apresentados requisitos que descrevem as principais funcionalidades e características do sistema a ser desenvolvido.

Capítulo 5

Desenho

Juntar este c
com a Análise

5.1 Diagrama de Implantação

Para além dos serviços funcionais descritos, a solução proposta deve, futuramente, ser exposta através de uma interface de acesso controlada e escalável. Desta forma, propomos a utilização de um *API Gateway* (Gateway de API) para agir como uma porta de entrada do serviço, controlando autorizações e acessos, e gerir o tráfego de pedidos feitos à *API*. Em paralelo, apresentamos a necessidade de um *Load Balancer* (balanceador de carga) que ajuda a distribuir tráfego entre múltiplas instâncias do serviço, assegurando maior disponibilidade e tolerância a falhas.

Adicionalmente, é sugerido o desenvolvimento de uma aplicação *Web* que permita a visualização da informação gerada por parte dos *OCEs*, onde estes poderão consultar o resumo do incidente pretendido, *tickets* relacionados e sugestões de mitigação.

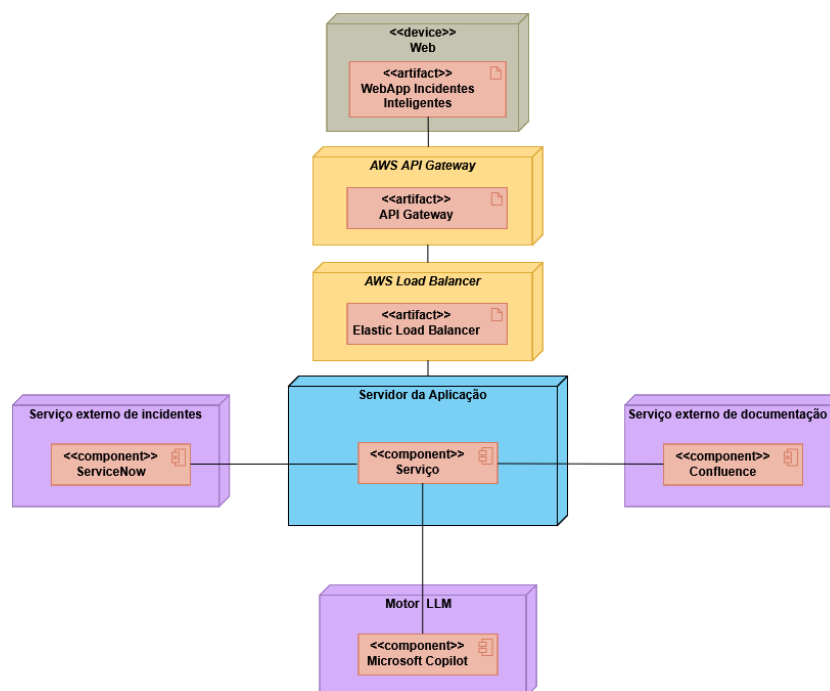


Figura 5.1: Diagrama de Implantação

5.2 Diagrama de Sequência

Rever e adicio
detalhes

Apresenta-se apenas um diagrama de sequência que representa um fluxo bem sucedido de recuperação e enriquecimento de contexto utilizado na análise de um incidente. O diagrama começa com a obtenção dos detalhes do incidente através do serviço de ITSM, sendo posteriormente utilizada uma chamada ao *LLM* para identificar incidentes relacionados e gerar uma pesquisa para a documentação do Confluence. Com base nessa pesquisa, o sistema obtém as páginas relacionadas com o serviço e o respetivo conteúdo.

De seguida, o conteúdo das páginas é enviado ao *LLM*, que identifica a documentação mais relevante para o incidente e resume a informação encontrada. Após esta etapa, são obtidos os incidentes relacionados e toda a informação recolhida é utilizada numa chamada final ao *LLM*, responsável por enriquecer a informação do incidente. O diagrama na Figura 5.2 permite visualizar a interação entre os principais componentes e compreender as diferentes etapas utilizadas para obter e combinar o contexto necessário para a análise do incidente.

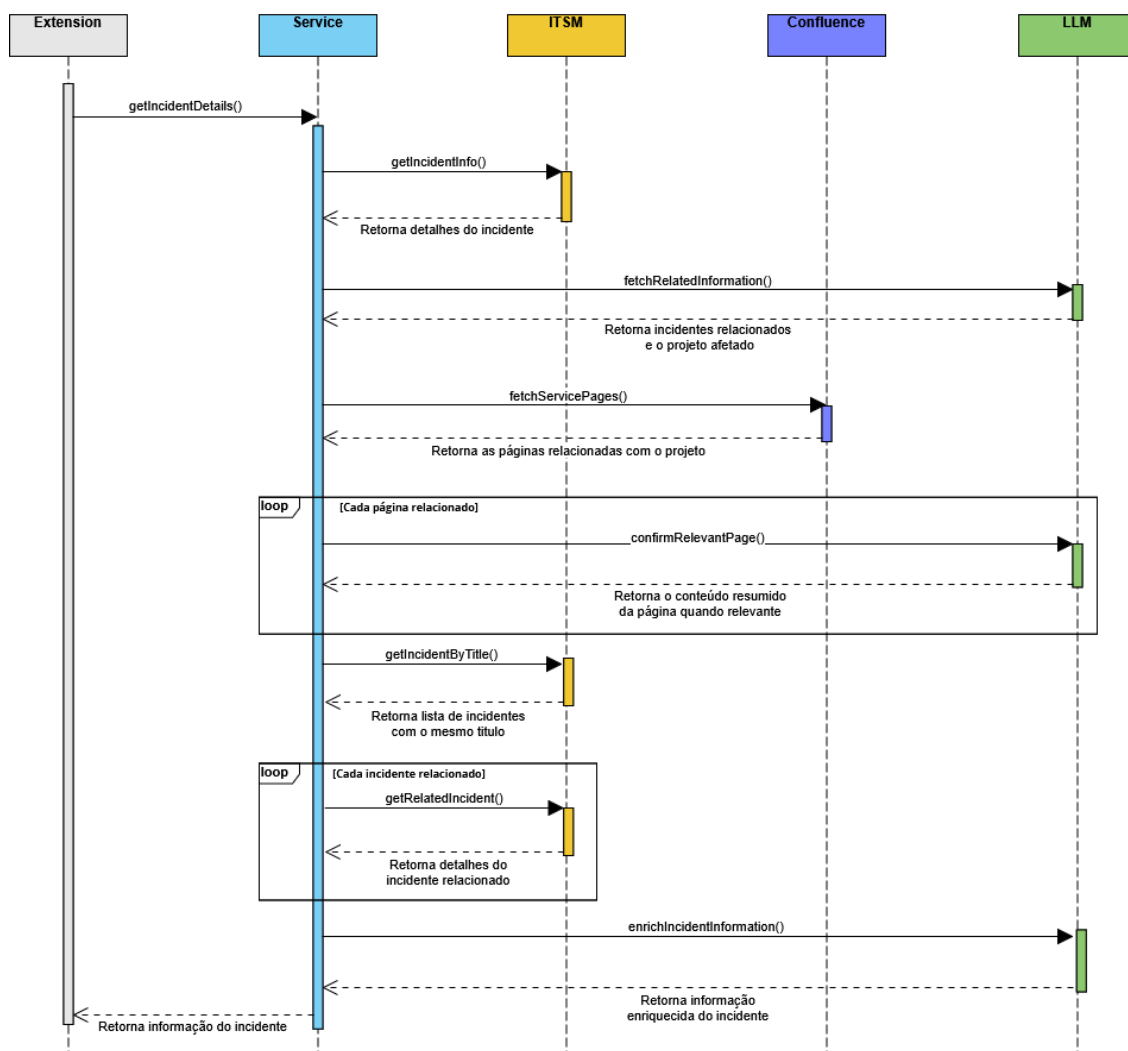


Figura 5.2: Diagrama de Sequência

5.3 Conclusão

Capítulo 6

Desenvolvimento

Neste capítulo são abordadas as principais decisões tomadas pelo autor durante a implementação das funcionalidades planejadas para o projeto. Para além das ferramentas e tecnologias utilizadas, é apresentada a metodologia seguida durante o desenvolvimento e a estrutura adotada para o *backend*. São também descritas as principais integrações realizadas e as opções tomadas na utilização do *LLM*, bem como algumas considerações relacionadas com os dados, custos e tempo de execução.

6.1 Stack Tecnológica

No início do projeto foi realizada uma análise sobre quais as melhores ferramentas e plataformas para o desenvolvimento do mesmo. Este capítulo apresenta as escolhas realizadas explicando os motivos de cada seleção.

6.1.1 Python

A linguagem Python foi escolhida para o desenvolvimento da solução principalmente pela sua forte utilização na área da Inteligência Artificial e pela variedade de bibliotecas disponíveis. Como o projeto envolve a utilização de *LLMs*, processamento de texto e implementação de uma arquitetura *RAG*, Python permite integrar estas diferentes componentes de forma relativamente simples.

Outro fator importante foi a facilidade de integração com serviços externos. A solução necessita de comunicar com o serviço de incidentes, com o motor *LLM* e com os restantes componentes utilizados na recuperação de informação. Python disponibiliza várias bibliotecas para comunicação através de APIs e para trabalhar com diferentes formatos de dados, o que facilita esta integração.

A linguagem também permite desenvolver e testar diferentes abordagens sem uma grande complexidade. Isto é particularmente importante neste projeto, uma vez que serão avaliadas diferentes configurações da solução, como a utilização de diferentes estratégias de recuperação e diferentes formas de fornecer contexto ao *LLM*. A existência de bibliotecas para testes, processamento de dados e avaliação dos resultados facilita ainda o desenvolvimento da componente experimental do trabalho.

Por fim, e ainda mais importante, Python apresenta uma sintaxe simples e uma comunidade bastante ativa, o que facilita a implementação das diferentes funcionalidades. Tendo em conta estas características e as necessidades da solução, considerou-se que Python seria uma escolha adequada para implementar o serviço e testar as diferentes abordagens propostas.

6.1.2 RAG

A utilização de *RAG* foi escolhida para permitir que o *LLM* tenha acesso ao histórico de incidentes no momento da análise, sem depender apenas do conhecimento que o modelo possui. Esta abordagem é particularmente adequada ao problema em estudo, uma vez que os incidentes estão constantemente a ser criados e atualizados. Desta forma, novos incidentes podem ser utilizados como contexto sem ser necessário voltar a treinar o modelo.

Para a recuperação dos incidentes optou-se por uma abordagem baseada na informação já disponível no *ServiceNow*, em vez da utilização de *embeddings*. A *API* do serviço externo permite obter a lista de incidentes da equipa e os respetivos dados, sendo possível aplicar filtros e, posteriormente, ordenar os resultados pelos incidentes mais recentes. Esta abordagem é suficiente para o contexto deste projeto, uma vez que grande parte dos incidentes analisados (98.41%) é gerada automaticamente pelos sistemas de monitorização e segue padrões semelhantes. Nestes casos, a utilização de uma pesquisa semântica através de *embeddings* não apresenta uma vantagem clara face à utilização dos dados estruturados já disponíveis.

Esta opção permite também manter o processo de recuperação mais simples e fácil de controlar. Em vez de criar e manter uma base de dados de *embeddings*, os incidentes são obtidos diretamente da fonte de informação existente e filtrados de acordo com as necessidades da análise. Os incidentes selecionados são depois utilizados como contexto para o *LLM*, que fica responsável por analisar a informação e gerar a resposta.

A escolha de *RAG* em vez de *fine-tuning* está também relacionada com a natureza dos dados. O *fine-tuning* permite adaptar o comportamento do modelo a uma determinada tarefa, mas não é a melhor opção quando o objetivo principal é fornecer informação que está constantemente a mudar. No caso dos incidentes, novos comentários e as respetivas resoluções surgem regularmente, o que obrigaria a atualizar o treino do modelo para acompanhar a informação mais recente. Com *RAG*, essa informação pode ser obtida diretamente da fonte de dados e utilizada pelo modelo durante cada análise, sem necessidade de novo treino.

Assim, a abordagem escolhida procura aproveitar a informação histórica já disponível no *ServiceNow*, mantendo o processo de recuperação simples e permitindo que o *LLM* tenha acesso a informação atualizada. O *fine-tuning* poderia ser utilizado como complemento no futuro, caso fosse necessário adaptar o comportamento ou o formato das respostas do modelo.

6.1.3 Langfuse

Para acompanhar a utilização dos *LLMs* foi utilizado o Langfuse, permitindo obter uma maior visibilidade sobre as operações realizadas durante cada pedido. A ferramenta permite acompanhar as diferentes chamadas ao modelo e os respetivos custos, tornando possível perceber quanto é gasto em cada etapa do processo de análise de um incidente.

Esta informação é importante para a avaliação da solução, uma vez que permite calcular o custo associado a cada pedido e identificar quais as operações que têm maior impacto no consumo. Desta forma, é possível não só acompanhar os custos da solução, mas também identificar oportunidades de otimização.

A utilização do Langfuse permite ainda que esta análise possa ser expandida no futuro. Para além do custo por pedido, poderão ser analisadas outras dimensões, como os custos

associados a diferentes utilizadores, sessões ou tipos de operação. Isto permite obter uma visão mais detalhada sobre a utilização do sistema e apoiar futuras decisões de otimização.

6.2 Metodologia de Implementação

Nesta secção, é demonstrada a metodologia escolhida para o desenvolvimento do projeto (*SDD*) assim como as razões por trás dessa escolha, as limitações, os métodos utilizados para combater os problemas causados pela metodologia e a forma como foi implementada.

6.2.1 Spec Driven Development

O desenvolvimento de software tem sido baseado em diferentes formas de organizar e controlar o processo de implementação. Com o aumento da complexidade dos sistemas e, mais recentemente, com a utilização de ferramentas de IA na programação, tornou-se cada vez mais importante definir de forma clara aquilo que se pretende desenvolver antes de iniciar a implementação. Quando o desenvolvimento começa diretamente pelo código, surgem, normalmente, problemas relacionados com requisitos pouco claros, decisões técnicas inconsistentes e diferenças entre o comportamento esperado e o comportamento implementado.

Esta metodologia coloca as especificações no centro do processo de desenvolvimento, sendo utilizadas como referência para definir e implementar as diferentes funcionalidades do sistema. Em vez de começar diretamente pela implementação, cada funcionalidade é primeiro descrita e detalhada através de uma especificação. A partir dessa documentação é definido o plano de implementação e, de seguida, as tarefas necessárias para desenvolver a funcionalidade.

Esta abordagem foi escolhida principalmente devido à utilização de IA como apoio ao desenvolvimento. Ao definir primeiro aquilo que o sistema deve fazer, é possível fornecer ao agente uma descrição mais clara do objetivo e das condições que a implementação deve cumprir. Desta forma, a utilização de IA não fica limitada à geração direta de código a partir de um pedido, mas passa a fazer parte de um processo mais estruturado.

Apesar da existência de um kit oficial de ferramentas (*spec-kit*), optou-se por aplicar *SDD* diretamente assim, a aplicação da metodologia, não ficou dependente da estrutura definida pela ferramenta. Por esse motivo, o projeto conta com uma série de ficheiros documentais, que ajudam tanto no processo de criação de especificações como também no processo de implementação.

A utilização desta abordagem permitiu também dedicar mais tempo ao pensamento crítico e à compreensão do negócio, em vez de focar principalmente na implementação do código. Desta forma, foi possível dar maior atenção às decisões e ao problema que a solução pretendia resolver.

6.2.2 Limitações

Apesar das vantagens desta abordagem, foram identificadas algumas limitações durante o desenvolvimento. Uma delas foi o *spec drift*, que ocorre quando são realizadas alterações no código sem que a respetiva especificação seja atualizada de forma prévia. Por exemplo, uma alteração feita através de *vibe coding*, como adicionar uma lógica de *retry* ao pedido feito ao *ServiceNow*, pode ser implementada diretamente sem passar pelo processo definido pelo *SDD*. Assim, o agente só segue esta abordagem quando essa disciplina é imposta

pelo utilizador, podendo uma instrução simples levar à implementação direta e deixar a especificação desatualizada.

Outra limitação está relacionada com o esforço adicional (*overhead*) associado a pequenas alterações. Mesmo uma alteração simples pode exigir a atualização da especificação, a implementação e a posterior validação. Embora este processo ajude a manter o código alinhado com as especificações, pode representar um esforço adicional para alterações de pequena dimensão.

6.2.3 Agentic SDD

Assim, foram criados agentes com o objetivo de apoiar o desenvolvimento e ajudar a garantir o cumprimento do processo definido pelo *SDD*. Dessa forma, um agente pode ser visto como um assistente que, para além de responder a instruções, consegue seguir um conjunto de regras, consultar informação disponível no projeto e realizar diferentes tarefas de acordo com o contexto fornecido. Para além de ajudarem na criação e atualização das *specs*, os agentes são instruídos a consultar informação adicional do projeto antes de realizar alterações. Esta informação encontra-se organizada em diferentes ficheiros, nomeadamente:

- **Template para as especificações** - Modelo base de uma especificação. Define os capítulos que todas as especificações devem conter assim como tópicos que devem ser abordados dentro de cada secção;
- **Arquitetura** - Descreve a arquitetura pretendida para o projeto. Define quais as camadas e como devem ser organizadas;
- **Requisitos** - Define os requisitos funcionais, não funcionais e de observabilidade do sistema;
- **Especificação da API** - Descreve as especificações da API a ser desenvolvida, definindo os parâmetros de entrada e saída do serviço.

Estes documentos foram o passo zero da implementação do projeto, servindo de base para iniciar a descrição das funcionalidades, objetivos pretendidos e a forma de os alcançar. Após essa definição foram criados os agentes que permitiriam aplicar o processo *SDD* ao desenvolvimento do projeto. Assim, os agentes criados foram:

- **Arquiteto de especificações** - Interpreta o contexto fornecido pelo desenvolvedor, define critérios de aceitação, cria/atualiza especificações de acordo com o modelo definido e atualiza outros documentos quando necessário. Nunca implementa código nem testes;
- **Engenheiro de Software** - Interpreta as especificações já criadas, cria tarefas orientadores, implementa código e testes e valida as implementações realizadas. Nunca altera especificações, quando deteta essa necessidade deve invocar o agente responsável;
- **Agente de validação** - Compara implementações com especificações, valida a cobertura de critérios de aceitação e pede correções quando necessário.

A utilização destes agentes, permite não só automatizar parte das tarefas associadas ao *SDD*, mas também orientar o desenvolvedor durante todo o processo garantindo que o contexto necessário é considerado antes da implementação.

6.2.4 Specs

As especificações foram organizadas em arquivos próprios dentro da pasta docs/specs/, seguindo o modelo definido no início do projeto, como demonstrado na figura 6.1. Este modelo permite manter uma estrutura semelhante entre as diferentes especificações e facilita a sua consulta e atualização ao longo do desenvolvimento.



Figura 6.1: Modelo Base de uma Especificação

Cada especificação contém a informação necessária para orientar o desenvolvimento da funcionalidade, incluindo o problema a resolver, os objetivos, os critérios de aceitação, as integrações com serviços externos, o plano de testes e as alterações necessárias. A especificação pode também ser atualizada ao longo do desenvolvimento sempre que sejam identificadas novas necessidades ou alterações ao comportamento esperado.

Todo este trabalho deve ser realizado com o uso do Arquiteto de especificações para que, no momento da criação das especificações, o agente saiba que documentos extra deve ter em conta para além do contexto dado pelo desenvolvedor.

Já o desenvolvimento foi dividido em diferentes fases, permitindo adicionar funcionalidades de forma gradual e validar cada etapa antes de avançar para a seguinte. A primeira fase esteve focada na criação da *API* e na primeira integração com o *LLM*. A segunda fase adicionou contexto de incidentes, observabilidade e integração com a documentação do *Confluence*. Por fim, a terceira fase contém uma funcionalidade que já foi especificada e preparada para implementação, mas que ficou fora do âmbito do trabalho desenvolvido.

A seguinte lista apresenta as especificações criadas ao longo do desenvolvimento, agrupadas por fase:

- **Fase 1**

- *US-1.1: Disponibilização da API de detalhes do incidente*
- *US-1.2: Obtenção dos dados base do incidente a partir do cliente*
- *US-1.3: Integração do enriquecimento com LLM no GET /incident/details*
- *US-1.4: Registo do custo de utilização do LLM e do tempo de resposta*
- *US-1.5: Avaliação manual entre versões*

- **Fase 2**

- *US-2.1: Adição da análise de logs e de observabilidade mais detalhada*
- *US-2.2: Implementação de rastreabilidade distribuída com Langfuse e OpenTelemetry*
- *US-2.3: Integração do contexto de incidentes relacionados no enriquecimento com LLM*
- *US-2.4: Pesquisa alternativa por incidentes recentes da equipa quando não são encontrados incidentes com o mesmo título*
- *US-2.5: Utilização de um LLM para encontrar incidentes relacionados e documentação do serviço no Confluence*

- **Fase 3**

- *US-3.1: Integração da base de dados de logs para fornecer registos de transações relacionados como contexto para o LLM*

A organização das especificações por fases permitiu acompanhar a evolução da solução e manter o desenvolvimento dividido em partes mais pequenas. Desta forma, cada nova funcionalidade podia ser analisada, implementada e validada antes de avançar para a fase seguinte.

6.3 Estrutura

A arquitetura do projeto foi definida com base em princípios de *Domain-Driven Design* (DDD), adaptados às necessidades de um protótipo de apoio à análise de incidentes. O

objetivo foi manter a lógica relacionada com o negócio separada das tecnologias e serviços externos utilizados pelo sistema.

O domínio do projeto está centrado na análise e correlação de incidentes, incluindo a interpretação da informação dos tickets, a identificação de incidentes relacionados, a organização da informação e a geração de sugestões de mitigação. Desta forma, a lógica principal do serviço não fica diretamente dependente das *APIs* ou tecnologias utilizadas para obter ou processar esta informação.

Para garantir esta separação, o projeto foi dividido em três camadas principais, como demonstra a figura 6.2. A camada *domain* contém os modelos e regras de negócio, bem como as interfaces necessárias para comunicar com serviços externos. A camada *infrastructure* contém as implementações dessas interfaces e é responsável pela comunicação com sistemas como o *IT Service Management (ITSM)* e os serviços de *LLM*. Por fim, a camada *presentation* é responsável pela exposição da aplicação, nomeadamente através da *API REST*.

Domain não devia ter ficheiros Confluence

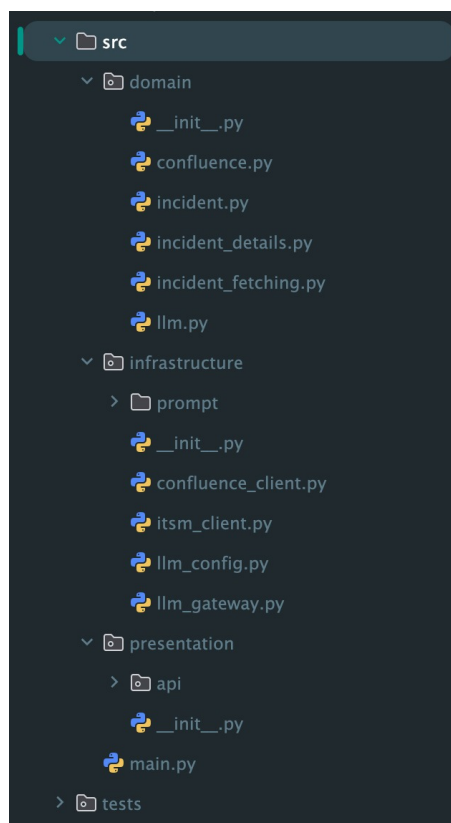


Figura 6.2: Modelo Base de uma Especificação

Esta organização foi escolhida também para dar resposta ao *RNF-02*, que define que o sistema deve permitir adicionar, substituir ou remover serviços externos sem impacto no restante serviço. Ao manter as integrações externas isoladas na camada de infraestrutura e ao definir os seus contratos no domínio, é possível substituir uma implementação sem alterar diretamente a lógica de negócio. Por exemplo, a integração com um determinado fornecedor

de *LLM* pode ser substituída por outra implementação sem necessidade de alterar as regras utilizadas na análise dos incidentes.

Assim, a estrutura adotada permite manter o núcleo da aplicação independente das tecnologias externas, facilitando futuras alterações e reduzindo o impacto das mudanças nas integrações. Ao mesmo tempo, mantém uma organização simples e adequada ao âmbito do projeto, evitando uma separação excessivamente complexa para as necessidades do protótipo.

6.4 Implementação

A aplicação integra diferentes fontes de informação e serviços externos, sendo necessário tratar e organizar essa informação antes de a disponibilizar ao *LLM*.

A implementação é apresentada de acordo com os principais componentes envolvidos no processo. Primeiro, é descrita a integração com o sistema *ITSM*, responsável pela obtenção do incidente principal e de informação sobre incidentes anteriores. De seguida, é apresentada a integração com o *Confluence*, utilizada para obter documentação adicional relacionada com o problema. Posteriormente, é explicada a integração com o *LLM*, incluindo a utilização dos diferentes *prompts*, a análise da informação recolhida e a geração das sugestões de mitigação. Por fim, são apresentadas algumas decisões tomadas durante a implementação, nomeadamente a utilização de *RAG*, o tratamento de dados sensíveis e o controlo dos custos e do tempo de execução.

6.4.1 Integração com ITSM

A integração com o *ITSM* (produto do *ServiceNow*) foi responsável por obter a informação necessária para a análise do incidente principal e por disponibilizar a informação de incidentes anteriores para que fosse utilizada de seguida pelo *LLM*. O processo foi dividido entre a obtenção do incidente principal e a recuperação de possíveis incidentes relacionados.

O processo começa com a obtenção do incidente a ser analisado. A informação devolvida pelo *ITSM* é, então, convertida para os modelos utilizados pelo domínio, permitindo que a restante aplicação trabalhe com uma estrutura independente da representação utilizada pelo sistema externo.

São selecionados apenas os campos necessários para a análise do incidente e para fornecer contexto adicional ao *LLM*. Desta forma, evita-se também a propagação de informação que não seja relevante ou que tenha sido identificada como sensível.

Após a obtenção do incidente principal, são realizadas pesquisas no *ITSM* para obter possíveis incidentes relacionados. Nesta fase, o objetivo não é determinar se os incidentes estão efetivamente relacionados, mas sim recolher um conjunto de incidentes que possa posteriormente ser analisado pelo *LLM*. São consideradas as referências no incidente principal através dos campos:

- `parent_incident`
- `description`
- `close_notes`
- `comments`

- `work_notes`
- `hold_reason`

Os incidentes encontrados são também identificados pelo seu número, permitindo evitar duplicados durante o processo.

A pesquisa é feita apenas a partir do incidente principal. Os incidentes obtidos nesta fase não são utilizados para procurar novos incidentes relacionados. Esta decisão evita que a pesquisa se torne recursiva e que sejam obtidos incidentes cada vez mais afastados do contexto do problema original.

É também realizada uma pesquisa por incidentes que tenham o mesmo `short_description` do incidente principal. Esta pesquisa permite encontrar ocorrências anteriores com um título semelhante, que podem conter informação útil sobre problemas e resoluções anteriores.

Quando a pesquisa por incidentes com o mesmo `short_description` não produz resultados, é utilizado um mecanismo de *fallback*. Neste caso, são obtidos incidentes mais recentes associados à equipa responsável, incluindo incidentes ativos e fechados. O objetivo deste mecanismo é garantir que o *LLM* recebe algum contexto histórico mesmo quando não existem incidentes anteriores com o mesmo título. Este *fallback* também permite ao *LLM* ter em conta falhas no serviço que possam estar relacionados ou mesmo ser a causa de outro incidente.

6.4.2 Integração com Confluence

O *Confluence* é utilizado como uma fonte adicional de informação para complementar os dados obtidos através do *ITSM*. O objetivo é encontrar documentação que possa ajudar a compreender melhor o problema identificado no incidente e que possa ser utilizada pelo *LLM* durante a análise final.

A pesquisa no *Confluence* não é realizada sobre todas as páginas disponíveis. Para limitar os resultados a informação relevante para o projeto, a pesquisa é restringida ao espaço referente desse produto. A partir da informação do incidente são procuradas páginas que possam estar relacionadas com o projeto. Quando uma página é encontrada, são também consideradas as suas subpáginas, uma vez que a informação sobre uma determinada funcionalidade ou componente pode estar distribuída por diferentes páginas da documentação.

Depois de identificadas as páginas, é obtido o seu conteúdo para que possa ser utilizado nas etapas seguintes. Nesta fase são recolhidas as informações necessárias para identificar cada página, desta forma, a aplicação consegue manter a referência à origem da informação que será posteriormente utilizada no contexto enviado ao *LLM*.

A pesquisa pode devolver páginas que, apesar de estarem relacionadas com o projeto ou com os termos utilizados na pesquisa, não sejam relevantes para o incidente em análise. Por esse motivo, a informação obtida do *Confluence* é analisada com o apoio do *LLM*, que determina quais as páginas que apresentam informação útil para o problema atual.

Após a seleção das páginas relevantes, o seu conteúdo é integrado com a restante informação recolhida durante o processo. Assim, o contexto final utilizado pelo *LLM* pode incluir a informação do incidente principal, os incidentes relacionados e a documentação relevante do *Confluence*. Para além do conteúdo, é mantida a referência para cada página selecionada, permitindo identificar posteriormente a documentação utilizada e apresentar ao utilizador a respetiva página como fonte de informação.

6.4.3 Integração com LLM

Os *LLM* são modelos de inteligência artificial treinados com grandes quantidades de dados textuais, sendo capazes de compreender e gerar texto em linguagem natural. Neste contexto, o *LLM* foi utilizado para analisar a informação recolhida dos diferentes sistemas e transformá-la em conhecimento útil para a análise do incidente. Em vez de utilizar o modelo apenas para gerar uma resposta a partir do incidente principal, a aplicação utilizou-o em diferentes momentos do processo, permitindo realizar tarefas específicas antes de devolver a resposta final.

A utilização do *LLM* foi dividida em três chamadas distintas. Esta divisão foi feita para evitar concentrar toda a informação e todas as tarefas numa única chamada, o que resultaria num *prompt* maior e mais difícil de controlar. Na primeira chamada, o modelo analisa o incidente principal e a informação dos incidentes candidatos obtidos através do *ITSM*, identificando quais apresentam uma relação relevante com o problema atual e qual o projeto ou componente afetado. Desta forma, a aplicação não assume que todos os incidentes obtidos durante a pesquisa são realmente relacionados, deixando essa decisão para o modelo. Na segunda chamada, o *LLM* analisa a documentação obtida através do *Confluence* e determina quais as páginas que contêm informação relevante para o incidente. A terceira chamada corresponde à análise final, onde são combinadas as informações selecionadas nas etapas anteriores para produzir as sugestões de mitigação.

Para controlar o comportamento do modelo nestas diferentes fases, foram utilizados *prompts* específicos. Um *prompt* corresponde à informação e às instruções fornecidas ao *LLM* para indicar o que deve ser analisado e qual o resultado esperado. A definição e melhoria destes *prompts* enquadra-se no conceito de *Prompt Engineering*, que consiste na criação e ajuste das instruções fornecidas ao modelo de forma a obter respostas mais consistentes e adequadas ao objetivo pretendido. Neste projeto, os *prompts* foram sendo ajustados ao longo do desenvolvimento, tendo em consideração não só a informação disponível, mas também a forma como os resultados deveriam ser apresentados e utilizados pelas etapas seguintes.

O primeiro *prompt* é responsável pela análise dos incidentes candidatos. O modelo recebe a informação dos campos não sensíveis do incidente principal, sendo instruído a identificar, nesses campos, incidentes relacionados com o problema atual. Nesta mesma etapa é também identificado o projeto ou componente afetado, informação que será utilizada posteriormente para orientar a análise e apresentar sugestões. A pesquisa de incidentes é feita apenas a partir do incidente principal, não sendo realizada uma nova pesquisa com base nos incidentes identificados como relacionados. Esta decisão evita uma expansão sucessiva da pesquisa, na qual um incidente poderia levar à obtenção de outros incidentes apenas por estarem relacionados entre si, aumentando a quantidade de informação enviada ao modelo e o risco de incluir informação pouco relevante.

O segundo *prompt* é utilizado para analisar a documentação obtida através do *Confluence*. A pesquisa inicial pode devolver páginas que estejam relacionadas com os termos utilizados, mas que não contenham informação útil para o problema específico. O *LLM* recebe, por isso, o conteúdo das páginas encontradas e seleciona aquelas que apresentam informação relevante para o incidente. O objetivo desta etapa não é gerar uma resposta final, mas reduzir a documentação disponível à informação que poderá contribuir para a análise seguinte.

A última chamada recebe a informação selecionada nas etapas anteriores, incluindo os dados do incidente principal, os incidentes identificados como relacionados e a documentação

considerada relevante. A preparação deste *prompt* foi uma das partes que sofreu mais alterações durante o desenvolvimento, uma vez que pequenas alterações na forma de apresentar a informação ou nas instruções dadas ao modelo podiam influenciar o resultado obtido. O objetivo foi chegar a um formato que permitisse ao *LLM* compreender facilmente o contexto e, ao mesmo tempo, produzir uma resposta que pudesse ser utilizada diretamente pela equipa de suporte.

A informação enviada ao *LLM* é organizada de forma estruturada, separando os diferentes elementos que fazem parte do contexto. O incidente principal é apresentado como o problema que está a ser analisado, enquanto os incidentes relacionados e a documentação do *Confluence* funcionam como informação adicional para ajudar na análise. Esta separação permite ao modelo distinguir a situação atual do histórico e da documentação utilizada como apoio. A aplicação também limita a quantidade de informação enviada ao modelo, evitando incluir dados que não sejam necessários para a análise e contribuindo para manter o custo e o tempo de execução dentro dos valores definidos para o sistema.

Com base neste contexto, a chamada final gera sugestões de mitigação para o incidente. As sugestões são construídas tendo em consideração não só o incidente atual, mas também as soluções registadas em incidentes anteriores e a informação existente na documentação selecionada. Desta forma, o objetivo não é simplesmente solicitar ao *LLM* que proponha uma solução genérica, mas fornecer-lhe informação relacionada com situações que já ocorreram e com o funcionamento do projeto afetado. A resposta produzida pelo modelo pode assim apresentar ações de mitigação que sirvam de apoio à equipa durante a resolução do incidente.

Por fim, a integração com o *LLM* é acompanhada através do *Langfuse*. Esta ferramenta permite registar e acompanhar as diferentes chamadas realizadas durante uma análise, incluindo informação sobre o tempo de execução e os custos associados a cada pedido. Como o processo utiliza várias chamadas ao *LLM*, esta observabilidade permite analisar cada operação individualmente e perceber quais as etapas que apresentam maior custo ou tempo de execução. Estes dados podem também ser utilizados posteriormente para avaliar o comportamento do sistema e identificar possíveis oportunidades de otimização.

Acrescentar imagem com traces do LangFuse e referir no texto acima

6.4.4 Considerações de Implementação

RAG vs Fine Tuning

Para o projeto foi escolhida a utilização de *RAG*, devido sua à natureza dinâmica da informação usada durante a análise. Os incidentes existentes no *ITSM* estão constantemente a ser alterados com a entrada de novos registos e, em particular, as informações relacionadas com a resolução dos incidentes podem mudar ao longo do tempo. O mesmo acontece com a documentação disponível no *Confluence*. Com o uso de *RAG*, esta informação pode ser obtida no momento da análise e fornecida ao modelo como contexto, sem ser necessário alterar o modelo sempre que são adicionados novos dados. No caso do *Fine Tuning*, os novos dados teriam de ser novamente utilizados num processo de treino para que fossem incorporados pelo modelo.

Para o caso dos incidentes, são recuperados do *ITSM* possíveis candidatos a incidentes relacionados com o incidente principal. Estes dados são posteriormente analisados pelo *LLM*, que determina quais são realmente relevantes para o problema em análise. A informação dos

incidentes selecionados, especialmente as descrições e as notas de resolução, pode fornecer contexto sobre problemas semelhantes e sobre as soluções utilizadas anteriormente.

Do mesmo modo, o *RAG* foi utilizado para a documentação, visto que, é algo frequentemente alterado com o surgimento de novas funcionalidades ou alteração das mesmas. Neste caso, são pesquisadas páginas no *Confluence* que possam estar relacionadas com o incidente e, após a sua recuperação, o modelo seleciona a informação considerada relevante. Esta informação é posteriormente utilizada como parte do contexto da análise final.

Dentro da abordagem de *RAG*, foi ainda considerada a utilização de uma pesquisa baseada em vetores onde, neste caso, os documentos seriam convertidos em *embeddings* e armazenados numa base de dados vetorial, permitindo recuperar informação com base na sua similaridade semântica. Apesar de esta abordagem ser útil em cenários onde existe uma grande quantidade de informação não estruturada, considerou-se uma opção mais simples.

A escolha de uma abordagem estrutural, permite também evitar a necessidade de manter uma infraestrutura adicional para criação, armazenamento e atualização de *embeddings*. Esta decisão é particularmente relevante devido à frequência com que os incidentes são criados e à necessidade de utilizar informação recente, sobretudo quando se pretende aproveitar as soluções de mitigação registadas em incidentes anteriores. Assim, a solução utiliza *RAG* através dos mecanismos de pesquisa existentes no *ITSM* e no *Confluence*, recuperando a informação no momento da análise em vez de a incorporar diretamente no modelo através de *Fine Tuning*.

Dados Sensíveis

A informação obtida através do *ITSM* pode conter dados sensíveis que não são necessários para a análise realizada pelo serviço. Como definido anteriormente na tabela 4.2, estes campos são identificados e removidos logo na camada de infraestrutura, no momento em que os dados são recebidos do sistema externo. Desta forma, a informação sensível não é transmitida para as restantes camadas da aplicação.

Esta decisão permite que o domínio trabalhe apenas com a informação necessária para a análise do incidente. Como os dados sensíveis são removidos antes de chegarem ao domínio, estes não fazem parte das entidades utilizadas pela lógica de negócio e, consequentemente, não podem ser utilizados nas chamadas ao *LLM* ou registados nos mecanismos de *logging* da aplicação. A proteção dos dados é assim realizada na entrada do sistema, em vez de depender de verificações individuais em cada etapa posterior do processamento.

Com esta abordagem é possível cumprir o *RNF-01*, presente na tabela 4.4, que define que o sistema não deve expor informação confidencial dos incidentes a utilizadores ou serviços que não tenham autorização para a consultar.

Custos e Tempo de Execução

Durante o desenvolvimento existiu também a preocupação de acompanhar o custo e o tempo de execução de cada análise. Estes valores são monitorizados através do *Langfuse*, que permite acompanhar as diferentes chamadas realizadas ao *LLM*, mas também são disponibilizados na resposta de cada pedido. Desta forma, é possível consultar diretamente o custo e o tempo associados a uma análise, permitindo avaliar o impacto das diferentes configurações utilizadas pelo sistema.

Para controlar a quantidade de informação enviada para o *LLM*, foram definidos vários parâmetros configuráveis. Um dos principais parâmetros corresponde ao limite máximo de *tokens* que podem ser enviados ao modelo. Este limite permite evitar, que um aumento inesperado da quantidade de informação recolhida resulte num pedido com uma dimensão excessiva e, num consequente aumento do custo e do tempo de execução. O modelo utilizado pelo sistema também é configurável, permitindo escolher entre diferentes modelos de acordo com o equilíbrio pretendido entre qualidade das respostas, custo e latência.

A quantidade de informação recuperada das fontes externas é igualmente controlada através de configurações. No caso do *ITSM*, é possível definir o número máximo de incidentes do histórico que podem ser obtidos e utilizados durante a análise. Esta configuração permite manter uma quantidade suficiente de histórico para que o *LLM* tenha contexto sobre problemas anteriores, sem enviar informação desnecessária. De forma semelhante, é possível definir o número máximo de páginas a obter do *Confluence*, evitando que uma pesquisa devolva uma quantidade excessiva de documentação para uma única análise.

Estas configurações permitem ajustar o nível de informação utilizado pelo sistema de acordo com as necessidades do projeto e com os recursos disponíveis. Em conjunto com o acompanhamento do custo e do tempo de execução, permitem avaliar o impacto das alterações realizadas e encontrar um equilíbrio entre a quantidade de contexto fornecida ao *LLM* e o custo de cada análise.

Com esta abordagem é possível cumprir o *RNF-03*, que define que o sistema deve limitar a quantidade de informação enviada para o *LLM*, de forma a manter o custo de cada análise dentro de valores aceitáveis.

6.5 Conclusão

Com este capítulo, foi possível verificar as principais ferramentas utilizadas, a metodologia aplicada e as decisões implementadas pelo autor do estudo, sendo que estas escolhas tiveram como principal objetivo construir uma solução simples, flexível e adequada ao problema identificado. A implementação permitiu também perceber a importância de separar as diferentes responsabilidades da aplicação e de manter as integrações externas isoladas da lógica principal do sistema.

Capítulo 7

Validação de Resultados

TODO

7.1 Metodologia

A metodologia de avaliação, considerava, inicialmente, métricas como a *ROUGE* e a *Precision@k*, utilizadas em trabalhos semelhantes encontrados durante o estado da arte. No entanto, no decorrer da preparação da avaliação foram identificadas algumas limitações na utilização destas métricas para o contexto deste projeto.

No caso da métrica *ROUGE*, a comparação é baseada na sobreposição de palavras entre a resposta gerada e a *golden reference*. Esta analogia, pode não representar corretamente a qualidade de uma resposta gerada por um *LLM*, uma vez que duas respostas podem transmitir a mesma informação utilizando palavras ou estruturas diferentes. Dessa forma, optou-se por utilizar uma abordagem de *LLM-as-a-judge*, onde um segundo modelo de *LLM*, avalia a resposta gerada tendo em conta a *golden reference* definida.

Na implementação da abordagem *LLM-as-a-judge*, foi desenvolvido um prompt para guiar e ajudar o juiz (*LLM*), na avaliação dos resumos e sugestões. No texto enviado ao modelo, foi definida a rubrica de avaliação, presente na tabela 7.1, que nos permitiu ter valores para avaliar e comparar no final.

Avaliação	Descrição
1	Completamente irrelevante
2	Pouco relevante e maioritariamente incorreto
3	Parcialmente relevante e correto mas falta elementos chave
4	Maioritariamente correto, com pequenas diferenças e informação extra irrelevante
5	Completamente correto e significado exatamente igual

Tabela 7.1: Campos Utilizados do Dataset

A *Precision@k* também não foi considerada adequada para a avaliação das sugestões de mitigação. Esta métrica, seria mais indicada caso existissem várias sugestões consideradas corretas na *golden reference*. Neste contexto, foi definida apenas uma sugestão considerada ideal para cada incidente, pelo que se pretende principalmente avaliar se essa sugestão aparece nos primeiros lugares da resposta. Assim, optou-se pela utilização do *Mean Reciprocal Rank* (*MRR*), que permite avaliar a posição em que a sugestão esperada é apresentada.

Para a avaliação dos incidentes relacionados e das páginas relacionadas, foi utilizada a cobertura (ou *recall@k*), uma vez que o objetivo é verificar se os elementos considerados

relevantes estão presentes nos resultados apresentados. Para além da qualidade dos resultados, foram também avaliados o custo e a latência da solução, considerando a média dos valores obtidos nos diferentes pedidos realizados.

Para realizar os testes, foi elencado um conjunto de quinze incidentes que, juntos, formam a *golden reference*, um exemplo pode ser visto na figura 7.1. Os sete primeiros incidentes, foram selecionados manualmente de forma a representar diferentes situações relevantes para a avaliação, e, os restantes oito foram selecionados aleatoriamente a partir do *dataset*. Desta forma, procurou-se combinar casos específicos, escolhidos para testar comportamentos conhecidos da solução, com casos selecionados de forma aleatória.

```
{
  "incident_id": "INC-002",
  "reference_summary": "In EMEA PROD, a customer refers that his car shows the correct location on the car display, but the address on the APP is incorrect.",
  "reference_mitigation_suggestions": [
    {
      "investigation": "Validate the referred coordinates to check what address is being returned by the service.",
      "mitigation": "If the issue is provider related, contact the provider to fix the issue."
    }
  ],
  "reference_related_incidents": [
    "INC-",
    "INC-",
    "INC-",
    "INC-"
  ],
  "reference_related_pages": [],
  "notes": "Manual: Car location is correct but is address on the APP is incorrect."
},
```

Figura 7.1: Exemplo de uma *Golden Reference*

Para definir cada *golden reference*, foi estabelecido um resumo simples em conjunto com a equipa atribuída ao incidente. Foi também acompanhado o processo de resolução, de forma a ser possível indicar a sugestão de investigação e mitigação utilizado para resolver efetivamente o incidente em questão.

Para a avaliação dos resumos, sugestões, elementos relacionados, custo e latência, cada um dos quinze incidentes foi analisado dez vezes utilizando o modelo *openai/gpt-5.6-terra*. Os resultados, obtidos comparando com as respetivas *golden references* e a repetição das análises permitiram reduzir o impacto das diferenças entre respostas produzidas pelo modelo e obter uma avaliação mais consistente.

Foi também realizada uma comparação entre diferentes modelos de *LLM*, onde foi utilizado o *openai/gpt-5.6-terra*, o *openai/gpt-5.4-mini*, o *claude-sonnet-5* e o *gemini-3.8-flash*. Para esta comparação foram realizados dois testes por incidente e por modelo, utilizando a mesma *golden reference* de quinze incidentes e permitindo observar diferenças entre os diferentes modelos.

7.2 Análise dos resultados

As seguintes análises, foram um resultado de testes realizados contra o conjunto de incidentes definido na *golden reference*. A escolha dos incidentes já foi referida anteriormente, mas é apresentada com mais pormenor na tabela 7.2.

Identificador	Descrição
INC001	Incidente manual com recurso ao mecanismo de <i>fallback</i>
INC002	Incidente manual com incidentes com o mesmo título
INC003	Incidente automático
INC004	Incidente com referência a outro incidente nos seus campos
INC005	Incidente com documentação relacionada
INC006	Incidente que deve ser redirecionado
INC007	Incidente genérico, sem documentação relacionada
INC008 a INC0015	Incidentes aleatórios distintos

Tabela 7.2: Incidentes na *Golden Reference*

7.2.1 Avaliação dos Resumos

Tendo por base, a rubrica de avaliação da tabela 7.1, definimos como objetivo, que os resumos gerados tivessem na sua maioria corretos e com todos os elementos chave. Estes, são os objetivos que permitem aos membros da equipa de suporte compreender o incidente, independentemente do nível de senioridade, do contexto do produto ou da hora em que o problema ocorre. Assim, a tabela 7.3 apresenta os critérios para a avaliação dos resumos gerados.

Métrica	Resultado
Classificação média pretendida	≥ 4.0
Percentagem de resumos com classificação ≥ 4.0	90%

Tabela 7.3: Critérios de Avaliação dos Resumos

Para a definição da classificação atribuída a cada resumo gerado, o *LLM-as-a-Judge*, utilizou a escala apresentada na tabela 7.1. Em cada iteração dos testes, o *LLM* comparava o resultado obtido na análise do serviço com a *golden reference* e atribuía um dos valores da escala.

A bateria de testes executada, contou com 150 análises, tendo em conta, que foram analisados quinze incidentes e cada um foi iterado dez vezes para evitar incoerências dos modelos de *LLM*. Dessa forma, foram obtido os resultados presentes na tabela 7.4, que demonstram que os critérios previamente definidos foram alcançados. Isto significa que, o serviço foi capaz de gerar, na sua maioria (98.6%), resumos maioritariamente corretos, com pequenas diferenças (nível 4).

Métrica	Resultado
Classificação 3	2 (1,3%)
Classificação 4	119 (79,3%)
Classificação 5	29 (19,3%)
Percentagem de resumos com classificação ≥ 4.0	98.6%
Classificação média	4,18

Tabela 7.4: Avaliação dos Resumos Gerados

7.2.2 Avaliação das Sugestões de Mitigação

Tal como na avaliação dos resumos, foram definidos os objetivos para classificar o sucesso, ou não, do serviço na criação de sugestões de mitigação. Assim, a tabela 7.5 apresenta os o *MRR* pretendido, evidenciando, o objetivo de ter a sugestão da *golden reference* nas posições mais elevadas, surgindo no topo da lista. Apresenta ainda, o objetivo de ter a sugestão real no top-1, em 75% ou mais, dos casos.

Métrica	Resultado
<i>MRR</i> médio pretendido	≥ 0.80
Top-1 pretendido	$\geq 75\%$

Tabela 7.5: Critérios de Avaliação das Sugestões

Observou-se, no entanto, que estes objetivos não foram totalmente cumpridos, tal como demonstra a tabela 7.6. O *MRR* médio obtido foi de 0,472, sendo que, em 40,7% dos casos a sugestão correspondente à *golden reference* encontra-se na primeira posição, aumentando para 55,3% quando são consideradas as três primeiras posições. Em 44,0% dos casos não foi encontrada qualquer correspondência entre a *golden reference* e as sugestões geradas.

Estes resultados devem-se, na sua maioria, ao facto de muitas das sugestões presentes nas *golden references* não terem correspondência direta com as sugestões fornecidas pelo serviço. Isto acontece porque as sugestões de referência correspondem a soluções reais para os problemas analisados. Mesmo existindo histórico de incidentes, uma única sugestão de investigação pode não ser suficiente para realizar uma análise completa dos incidentes de referência.

Métrica	Resultado
<i>MRR</i> médio	0,472
Top-1 (até posição 1)	61 (40,7%)
Top-2 (até posição 2)	74 (49,3%)
Top-3 (até posição 3)	83 (55,3%)
Top-4 (até posição 4)	84 (56,0%)
Sem correspondência	66 (44,0%)

Tabela 7.6: Avaliação das Sugestões Geradas

Se realizarmos a mesma análise, retirando os casos em que as sugestões não têm correspondência, verificamos que os resultados são mais positivos, visível na tabela 7.7. Cumpre-se assim o objetivo relacionado com o *MRR*, que evidencia que as sugestões com correspondência estão, na sua maioria, no topo das sugestões.

Para perceber o desempenho do serviço nos casos em que existe uma correspondência, foi realizada uma segunda análise, excluindo os casos em que não foi encontrada qualquer sugestão correspondente. Os resultados são apresentados na tabela 7.7. Nesta análise, o *MRR* médio aumenta para 0,842, cumprindo o objetivo apresentado na tabela 7.5. Para além disso, 72,6% das sugestões correspondentes encontram-se na primeira posição, também aproximando-se do valor alvo.

Métrica	Resultado
MRR médio	0,842
Top-1	61 (72,6%)
Top-2	74 (88,1%)
Top-3	83 (98,8%)

Tabela 7.7: Avaliação das Sugestões Geradas (Exclui não Correspondências)

Estes resultados permitem verificar que, quando o serviço gera uma sugestão correspondente à *golden reference*, esta tende a ser apresentada nas primeiras posições da resposta.

7.2.3 Avaliação dos Incidentes e Páginas Relacionadas

Os incidentes e páginas relacionadas são apresentados sobretudo como mecanismo de rastreabilidade, ou seja, para comprovar a origem da informação usada pelo serviço nas afirmações que produz. Por esse motivo, os objetivos definidos para esta avaliação são menos exigentes, evidenciado na tabela 7.8.

Métrica	Resultado
Cobertura média de incidentes relacionados pretendida	≥ 0.60
Cobertura completa de incidentes relacionados pretendida	$\geq 50\%$
Cobertura média de páginas relacionadas pretendida	≥ 0.60
Cobertura completa de páginas relacionadas pretendida	$\geq 50\%$

Tabela 7.8: Critérios de Avaliação dos Relacionados

Para esta análise, foi usada uma métrica de cobertura, isto significa que, em cada execução, mede-se a percentagem de itens da referência que foram encontrados na resposta. Foram assim, analisadas 150 execuções (quinze incidentes, dez iterações por incidente).

Nos incidentes relacionados, os resultados da tabela 7.9 mostram uma cobertura média de 0,917, isto é, o serviço encontra 91.7% dos incidentes relacionados que devia encontrar. Já a cobertura completa encontra-se nos 72,7% dos casos.

Métrica	Resultado
Cobertura média	0,917
Cobertura completa	109 (72,7%)
Cobertura parcial	38 (25,3%)
Sem correspondência (0,0)	3 (2,0%)

Tabela 7.9: Avaliação dos Incidentes Relacionados

Para as páginas relacionadas, os resultados da tabela 7.10 também são positivos, com uma cobertura média de 0,897 e cobertura completa em 85,3% dos casos.

Métrica	Resultado
Cobertura média	0,897
Cobertura completa	128 (85,3%)
Cobertura parcial	13 (8,7%)
Sem correspondência (0,0)	9 (6,0%)

Tabela 7.10: Avaliação das Páginas Relacionadas

Em resumo, os resultados mostram que o serviço consegue recuperar, na maioria das execuções, os itens relacionados esperados.

7.2.4 Avaliação da Qualidade do serviço

Nesta secção avaliamos o desempenho operacional do serviço em termos de tempo de resposta e custo por execução. Foram definidos os critérios da tabela 7.11, tendo em conta um cenário de cerca de 1000 incidentes por mês, média estipulada através da figura 4.1.

Métrica	Objetivo
Latência média pretendida	≤ 30 s
Custo médio por execução pretendido	≤ 0.05 USD
Custo mensal estimado (1000 incidentes)	≤ 50 USD

Tabela 7.11: Critérios de Avaliação de Latência e Custos

Os testes realizados para esta secção, basearam-se na mesma bateria de testes das secções anteriores tendo, portanto, utilizado os quinze incidentes de referência e iterando-os dez vezes.

Os resultados, demonstrados na tabela 7.12, mostram que os objetivos definidos para esta métrica não foram atingidos. A latência média observada foi de 68,53 segundos, quando o objetivo era 30 segundos, ou seja, cerca de 128% acima do valor alvo. Além disso, apenas 18,0% das execuções ficaram abaixo de 30 segundos. A mediana (78,53 segundos) e o P95 (91,69 segundos) confirmam que, na maioria dos casos, o tempo de resposta real está mais próximo de 1 minuto e meio do que de 30 segundos.

Métrica	Resultado
Latência média	68,53 s
Latência mediana	78,53 s
Latência P95	91,69 s
Execuções com latência ≤ 30 s	27 (18,0%)

Tabela 7.12: Avaliação de Latência

No custo, o comportamento foi semelhante tal como é comprovado na tabela 7.13. O custo médio por execução foi de 0,102 USD, acima da meta de 0,05 USD (cerca de 104% acima). Projetando para 1000 incidentes/mês, o custo estimado fica em 102,11 USD/mês, também acima da meta de 50 USD/mês. Isto indica que, no estado atual, o serviço ainda não cumpre os limites operacionais definidos para produção com esse volume.

Custo médio por execução	0,102 USD
Custo mediano por execução	0,117 USD
Custo P95 por execução	0,136 USD
Custo total da bateria de testes	15,32 USD
Custo mensal estimado (1000 incidentes)	102,11 USD

Tabela 7.13: Avaliação de Custos

Porém, através da análise apresentada no *LangFuse*, evidenciada na figura 7.2, podemos afirmar que a maioria do valor da latência e dos custos provêm das numerosas chamadas ao *Confluence*, para obter a documentação relevante ao incidente.

Observation	Depth	Start	Duration
>  llm.check_documentation_relevance (b2990f... 1 item	L1	0:13.360	2.6s
>  llm.check_documentation_relevance (5d010... 1 item	L1	0:15.947	3.2s
>  llm.check_documentation_relevance (a99c6... 1 item	L1	0:19.164	4.2s
>  llm.check_documentation_relevance (7c2b47... 1 item	L1	0:23.366	1.4s
>  llm.check_documentation_relevance (44fc34... 1 item	L1	0:24.783	3.3s
>  llm.check_documentation_relevance (4ef954... 1 item	L1	0:28.102	3.0s
>  llm.check_documentation_relevance (77d8bf... 1 item	L1	0:31.085	2.6s
>  llm.check_documentation_relevance (b31705... 1 item	L1	0:33.733	3.6s
>  llm.check_documentation_relevance (b628bf... 1 item	L1	0:37.350	3.7s
>  llm.check_documentation_relevance (a840af... 1 item	L1	0:41.026	2.9s
>  llm.check_documentation_relevance (0403c... 1 item	L1	0:43.925	3.3s
>  llm.check_documentation_relevance (bfb2ae... 1 item	L1	0:47.185	2.4s
>  llm.check_documentation_relevance (e6739... 1 item	L1	0:49.580	836ms
>  llm.check_documentation_relevance (64ff5a... 1 item	L1	0:50.416	4.3s
>  llm.check_documentation_relevance (73c49... 1 item	L1	0:54.753	2.5s
>  llm.check_documentation_relevance (57851b... 1 item	L1	0:57.227	1.8s
>  llm.check_documentation_relevance (6d11ca... 1 item	L1	0:59.040	964ms
>  llm.check_documentation_relevance (38b21e... 1 item	L1	1:00.004	2.5s
>  llm.check_documentation_relevance (70b0f4... 1 item	L1	1:02.488	1.6s
>  llm.check_documentation_relevance (8de5b... 1 item	L1	1:04.047	1.6s
>  llm.check_documentation_relevance (be881... 1 item	L1	1:05.610	2.1s
>  llm.check_documentation_relevance (c0d90... 1 item	L1	1:07.675	2.4s
>  llm.check_documentation_relevance (4f8285... 1 item	L1	1:10.066	2.2s
>  llm.check_documentation_relevance (1208c... 1 item	L1	1:12.269	3.8s
>  llm.check_documentation_relevance (cc39b... 1 item	L1	1:16.118	1.9s

Figura 7.2: Duração para Selecionar Documentação Relevante

Na análise apresentada anteriormente, a duração do processo de verificação de páginas relacionadas relevantes foi de 68.5 segundos, sendo que, a duração total da análise foi de 93.3 segundos. Assim, podemos concluir que 73% da duração das análises são alocadas para verificar que documentação pode ou não ser interessante para o contexto do incidente.

Para confirmar estes valores, foi realizada uma nova bateria de testes, mantendo os mesmo quinze incidentes de referência, e os valores obtidos estão apresentados na tabela 7.14. Posto isto, conseguimos afirmar que este processo de verificação de páginas relacionadas relevantes é de facto o maior causador na falha dos objetivos definidos nesta secção.

Configuração	Latência média (s)	Custo médio (USD)
Com <i>Confluence</i>	68,53	0,102
Sem <i>Confluence</i>	14,44	0,028
Diferença percentual	-78.9%	-72.3%

Tabela 7.14: Comparação de Latência e Custo sem *Confluence*

Para corrigir este problema, seria possível paralelizar estas chamadas, reduzindo assim a latência total deste processo. No entanto, esta abordagem não resolveria o problema do custo, que continuaria a ser elevado para uma equipa com cerca de 1000 incidentes por mês. Para reduzir este custo, seria possível juntar várias chamadas ao *LLM* numa única chamada, ou reduzir o número total de chamadas. Atualmente, cada documento é analisado através de uma chamada independente, o que aumenta o custo total. Ao juntar vários documentos numa mesma chamada, o custo não aumentaria necessariamente de forma proporcional ao número de documentos, permitindo reduzir o custo total do processo.

7.2.5 Avaliação dos Modelos LLM

Para avaliar o impacto da escolha do modelo nos resultados obtidos pela solução, foram realizados testes com diferentes modelos de *LLM*. Nestes testes, mantiveram-se as condições de avaliação comparáveis, sendo utilizados os mesmos incidentes presentes no conjunto de *golden references*, e onde cada incidente foi analisado duas vezes por cada modelo.

Foram considerados cinco modelos com diferentes características:

- **openai/gpt-5.6-terra**: utilizado como referência de qualidade;
- **openai/gpt-5.4-mini**: utilizado como referência de menor custo;
- **claude-sonnet-5**: utilizado devido às suas capacidades de compreensão e síntese;
- **gemini-3.8-flash**: utilizado devido à sua relação entre custo e latência.

Para cada modelo, foram comparados os resultados médios obtidos na avaliação dos resumos, das sugestões de mitigação, dos incidentes e páginas relacionadas e o custo e a latência médios de cada execução. Desta forma, pretende-se observar não só as diferenças na qualidade dos resultados produzidos, mas também o impacto da escolha do modelo nos recursos necessários para executar a solução.

7.2.6 Avaliação do Feedback da Equipa

7.3 Conclusão

Capítulo 8

Conclusão

TODO

8.1 Trabalho Realizado

8.2 Limitações

8.3 Trabalho Futuro

8.4 Apreciação Final

Bibliografia

- AIOps and Applied Intelligence | New Relic* (2026). en. url: https://newrelic.com/platform/applied-intelligence?has_gdpr=true (acedido em 06/01/2026).
- Chen, Yiru et al. (2024). «Dynamic Graph Neural Networks-Based Alert Link Prediction for Online Service Systems». Em: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*. ASE '23. Echternach, Luxembourg: IEEE Press, pp. 79–90. isbn: 9798350329964. doi: 10.1109/ASE56229.2023.00177. url: <https://doi.org/10.1109/ASE56229.2023.00177>.
- Chen, Yujun et al. (2020). «Identifying linked incidents in large-scale online service systems». Em: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2020. event-place: Virtual Event, USA. New York, NY, USA: Association for Computing Machinery, pp. 304–314. isbn: 978-1-4503-7043-1. doi: 10.1145/3368089.3409768. url: <https://doi.org/10.1145/3368089.3409768>.
- Elastic AI Assistant for Observability and Search | Elastic Docs* (2026). en. url: <https://www.elastic.co/docs/solutions/observability/observability-ai-assistant> (acedido em 06/01/2026).
- Goel, Drishti et al. (2024). «X-Lifecycle Learning for Cloud Incident Management using LLMs». Em: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. FSE 2024. event-place: Porto de Galinhas, Brazil. New York, NY, USA: Association for Computing Machinery, pp. 417–428. isbn: 979-8-4007-0658-5. doi: 10.1145/3663529.3663861. url: <https://doi.org/10.1145/3663529.3663861>.
- Jiang, Jiajun et al. (2020). «How to mitigate the incident? an effective troubleshooting guide recommendation technique for online service systems». Em: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2020. event-place: Virtual Event, USA. New York, NY, USA: Association for Computing Machinery, pp. 1410–1420. isbn: 978-1-4503-7043-1. doi: 10.1145/3368089.3417054. url: <https://doi.org/10.1145/3368089.3417054>.
- Jiang, Yuxuan et al. (2024). «Xpert: Empowering Incident Management with Query Recommendations via Large Language Models». Em: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ICSE '24. event-place: Lisbon, Portugal. New York, NY, USA: Association for Computing Machinery. isbn: 979-8-4007-0217-4. doi: 10.1145/3597503.3639081. url: <https://doi.org/10.1145/3597503.3639081>.
- Operações de IA* (2026). pt-BR. url: <https://aws.amazon.com/pt/cloudwatch/features/aiops/> (acedido em 06/01/2026).
- Roy, Devjeet et al. (2024). «Exploring LLM-Based Agents for Root Cause Analysis». Em: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. FSE 2024. event-place: Porto de Galinhas, Brazil. New York, NY, USA: Association for Computing Machinery, pp. 208–219. isbn: 979-8-4007-0658-5. doi: 10.1145/3663529.3663841. url: <https://doi.org/10.1145/3663529.3663841>.

- Using the Ops Guide agent / Rovo* (2026). en-US. url: <https://support.atlassian.com/rovo/docs/using-ops-guide/> (acedido em 06/01/2026).
- Zhang, Xuchao et al. (2024). «Automated Root Causing of Cloud Incidents using In-Context Learning with GPT-4». Em: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. FSE 2024. event-place: Porto de Galinhas, Brazil. New York, NY, USA: Association for Computing Machinery, pp. 266–277. isbn: 979-8-4007-0658-5. doi: 10.1145/3663529.3663846. url: <https://doi.org/10.1145/3663529.3663846>.

Apêndice A

Registos de incidentes

O registo de um incidente, é um conjunto de informações sobre um incidente em específico. Nesse registo é possível visualizar os detalhes desse mesmo *ticket*, como o identificador, o cliente, equipa ou alarme que originou o incidente, o serviço afetado, a sua prioridade e o estado atual (Figura A.1).

The screenshot shows a dark-themed user interface for an incident management system. At the top, there is a header bar with the title 'Incident' and a close button (upward arrow). Below the header, the form is organized into two columns. The left column contains fields for 'Number' (with value 'INC0001'), 'Caller *' (with value 'Technical identity for'), 'Originator group', 'Service', 'Service offering *', and 'Configuration item'. The right column contains fields for 'Channel' (with value 'Standard-UI'), 'State' (with value 'New'), 'Impact' (with value '3 - Low'), 'Urgency' (with value '2 - Medium'), 'Priority' (with value '4 - Low' and a radio button), 'Assignment group *', and 'Assigned to'. Each field has a search icon (magnifying glass) and an information icon (i) next to it.

Figura A.1: Detalhes do incidente

Associado a cada incidente, existe ainda um título, normalmente sucinto e objetivo, e uma descrição mais detalhada do problema. É nestes campos que o membro da equipa de suporta inicia a sua análise, sendo bastante importante conter descrições detalhadas, o que nem sempre é possível (Figura A.2).

Incident

Short description *

CloudWatch test environment test-service-api-ecs-task-stopped-unexpectedly ECS Task fail

Translate

Description *

CloudWatch test environment test-service-api-ecs-task-stopped-unexpectedly
ECS Task failed with stopCode ServiceSchedulerInitiated or EssentialContainerExited
Warning alert [Alert00000XXXXXX] .
Created on [AccountX].
Metric name is [test-service-api-ecs-task-stopped-unexpectedly-alarm] of type [] from data
source [AWS].

Total impacted services by the CI: [Product Y (Monitoring)] is 1.

Figura A.2: Detalhes do incidente

Para além dos campos originais do incidente, os desenvolvedores podem ainda deixar comentários extra que podem ser visíveis para o cliente ou apenas para as equipas de suporte. Esta abordagem é útil quando um incidente afeta várias equipas (Figura A.3).

Notes

Watch list

Work notes list

Additional comments (Customer visible)

Work notes

Figura A.3: Detalhes do incidente

Por fim, o incidente precise de notas de resolução onde os *OCEs* podem informar sobre os passos efetuados, origem do problema e outros comentários que achar necessário (Figura A.4).

Resolution Information

Resolution code

No resolution provided

Resolved by

—

Resolved

—

Resolution notes

This incident was due to high use of memory from 1 instance. Restarting all instances temporarily solved the issue.

Later the team will look with closer attention to the Root Cause.

Figura A.4: Detalhes do incidente